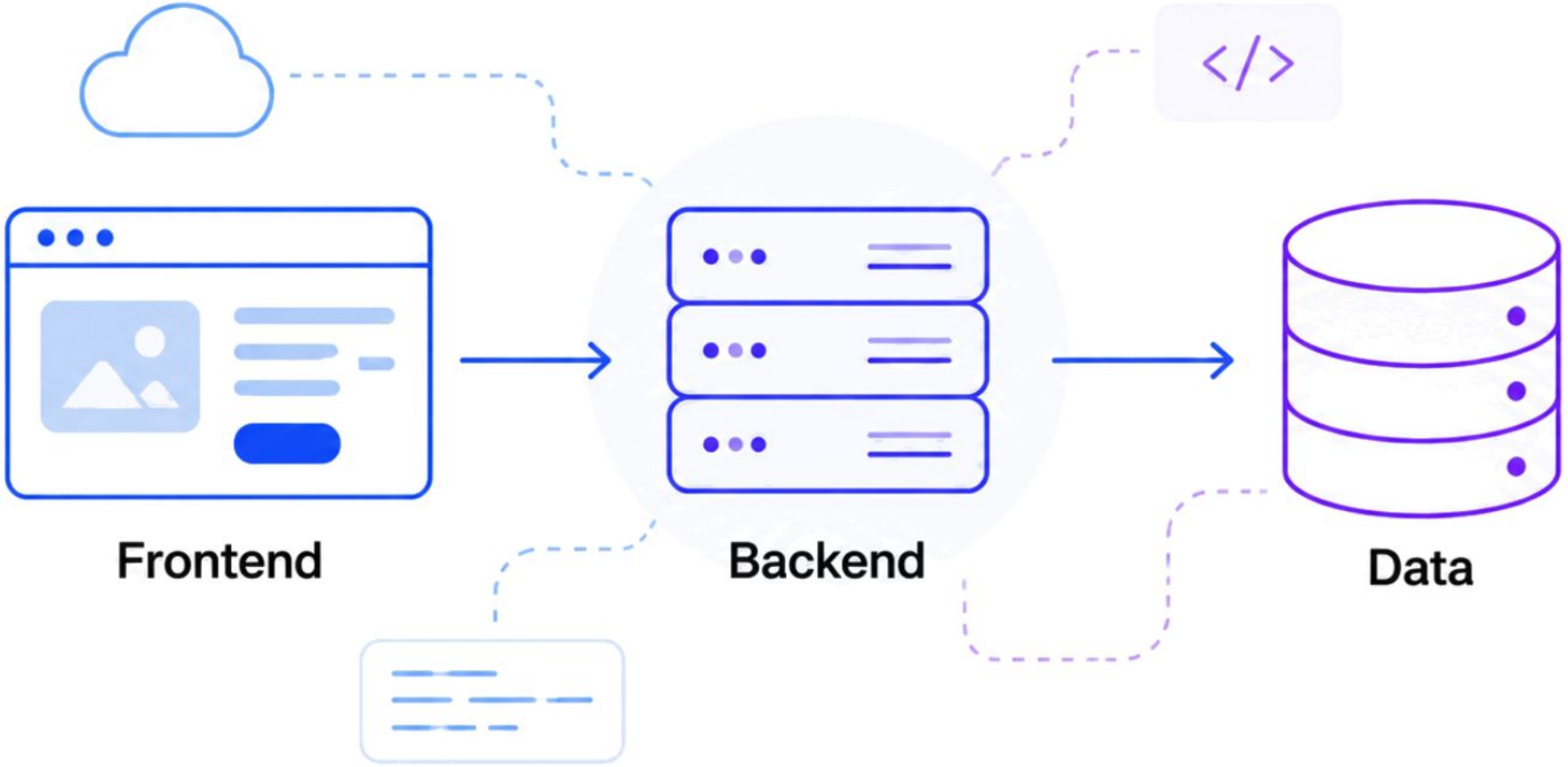
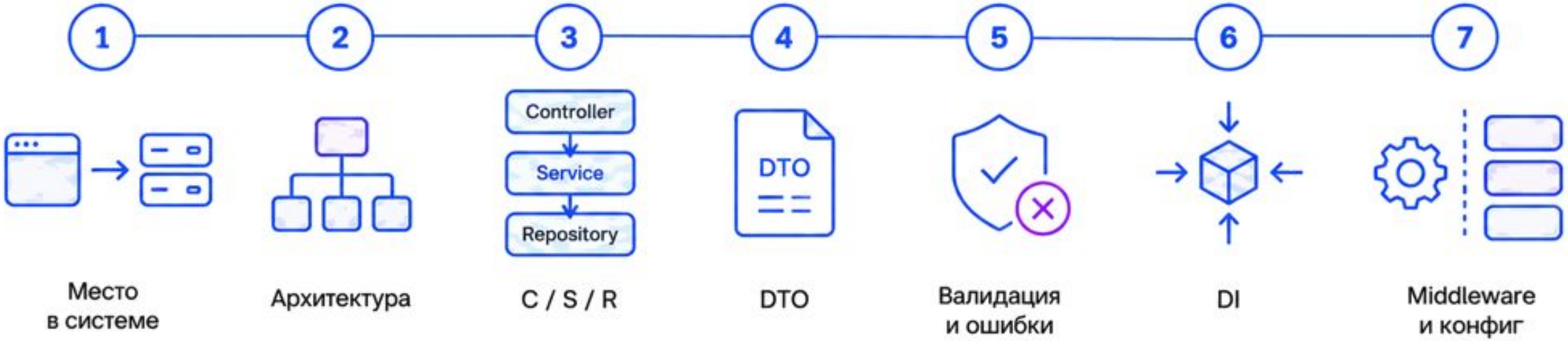
ИТМО

Backend как слой системы: архитектура и паттерны

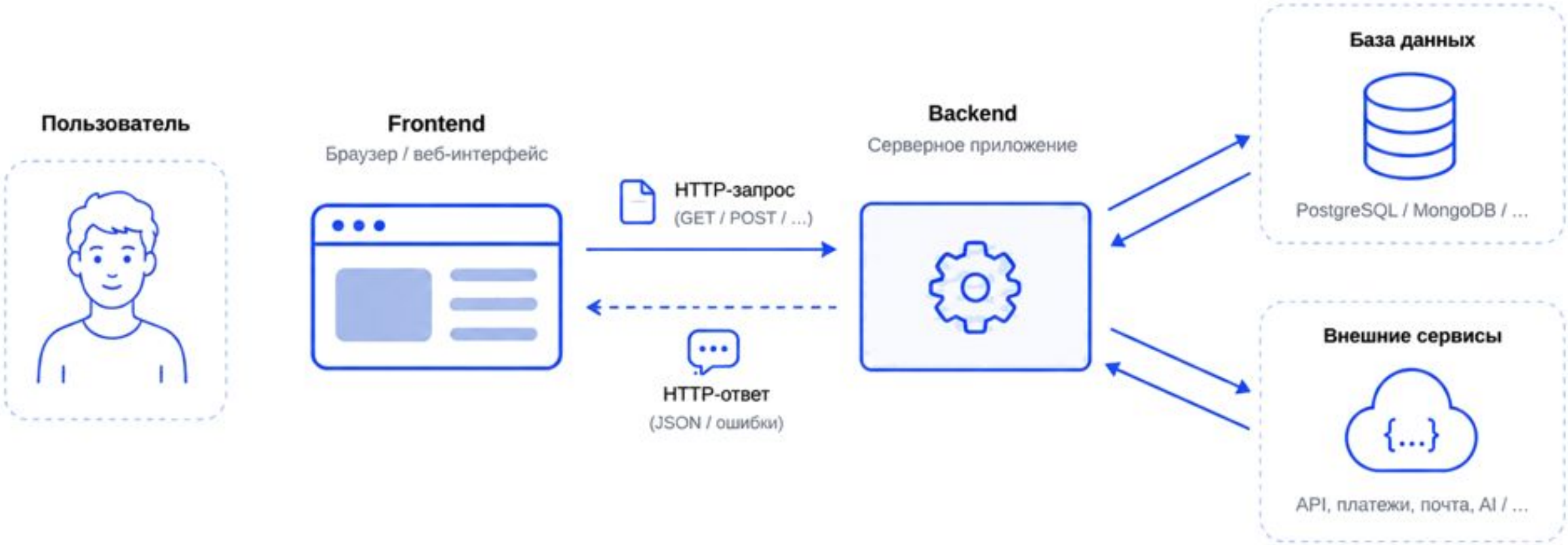
Backend как слой системы



План лекции


1
2
3
4
5
6
7
Controller
Service
Repository
DTO
Место в системе
Архитектура
C / S / R
DTO
Валидация и ошибки
DI
Middleware и конфиг

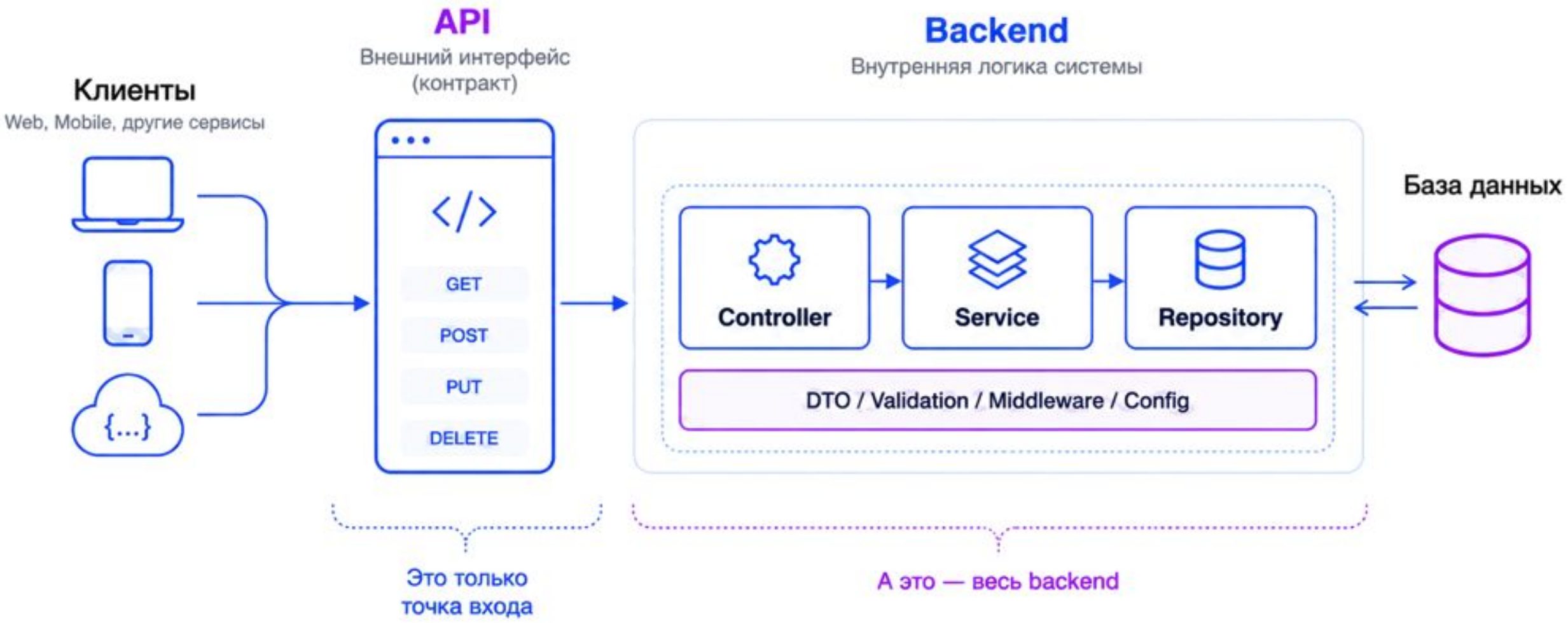
Где находиться backend



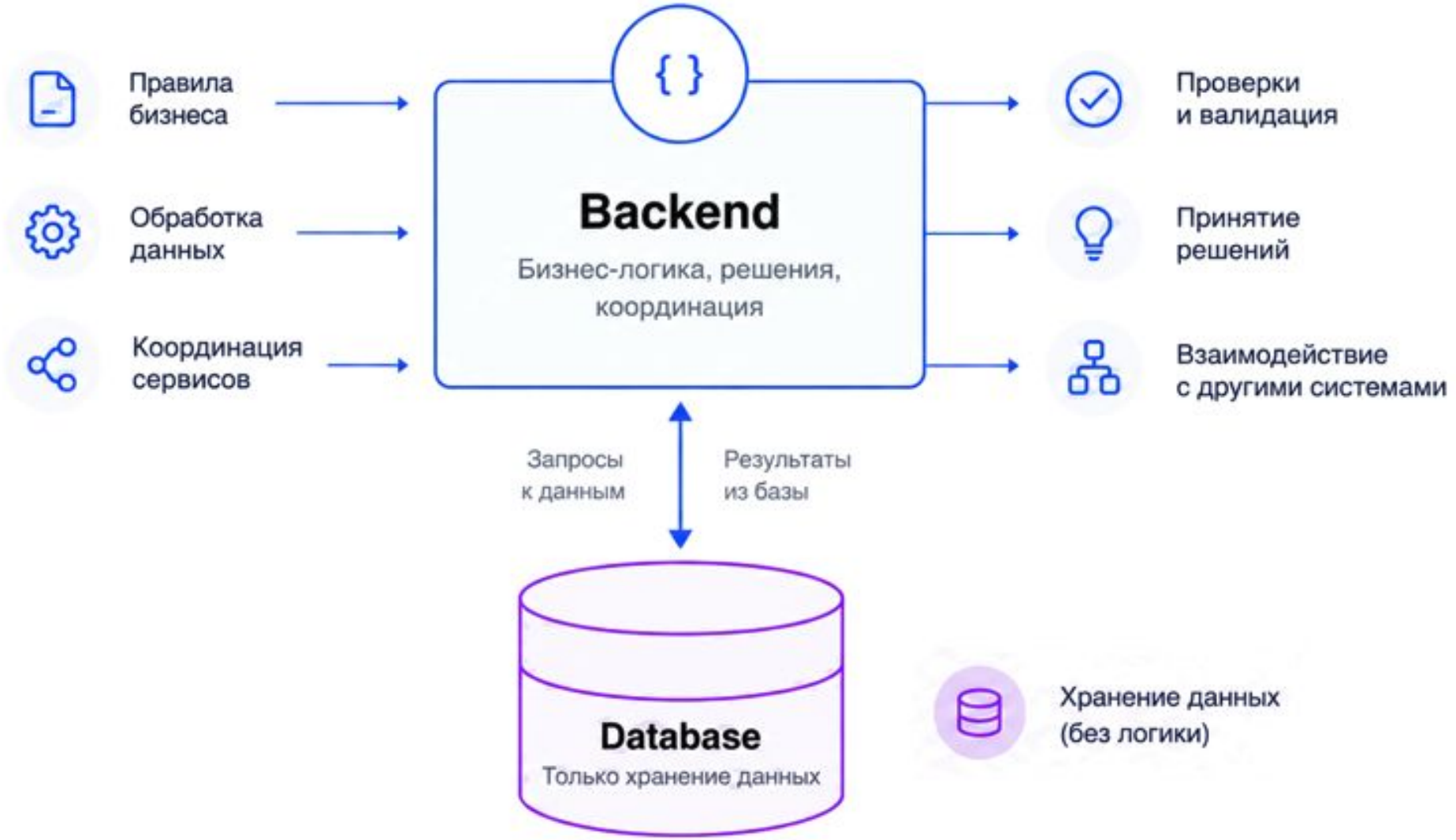
Актуальность



Backend ≠ API

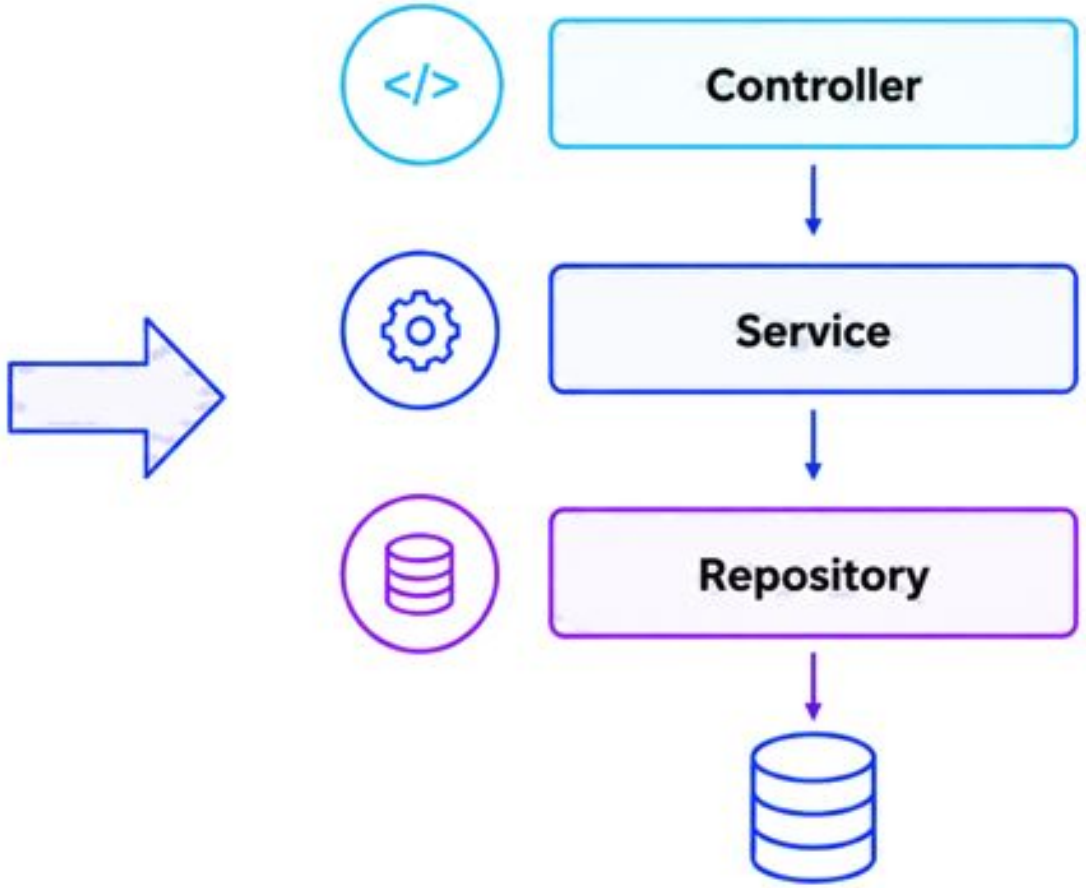

API
Внешний интерфейс
(контракт)
Backend
Внутренняя логика системы
Клиенты
Web, Mobile, другие сервисы
GET
POST
PUT
DELETE
Controller
Service
Repository
DTO / Validation / Middleware / Config
База данных
Это только
точка входа
А это — весь backend

Backend ≠ Database

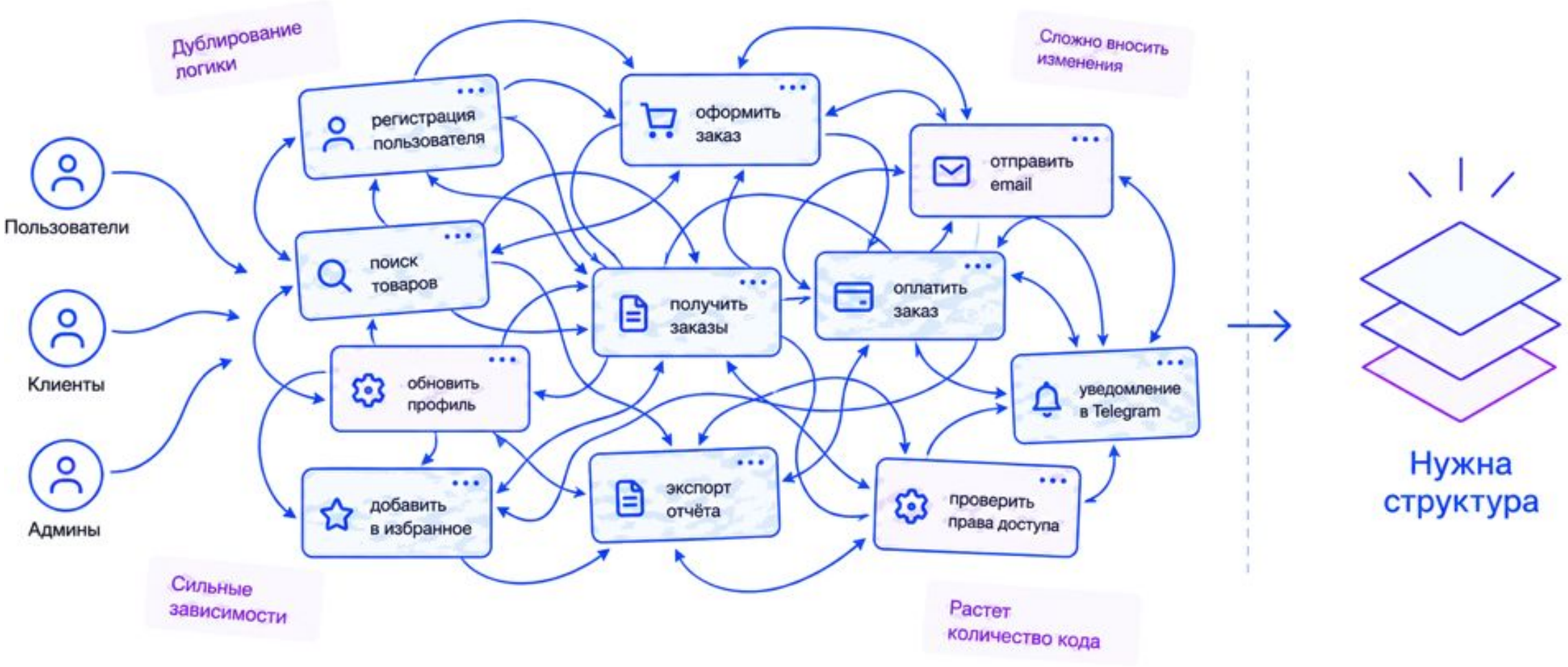


От простого обработчика к архитектуре

```
app.post('/users', async (req, res) => {
  // валидация
  const { name, email } = req.body;
  // бизнес-логика
  const user = await db.query(
    'INSERT INTO users (name, email) VALUES ($1, $2)',
    [name, email]
  );
  // ответ
  res.status(201).json(user);
});
```



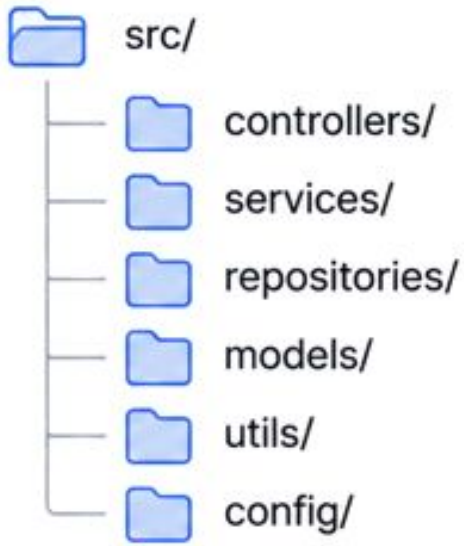
Проблемы роста приложения



Что такое архитектура

Просто папки

Видим только структуру файлов



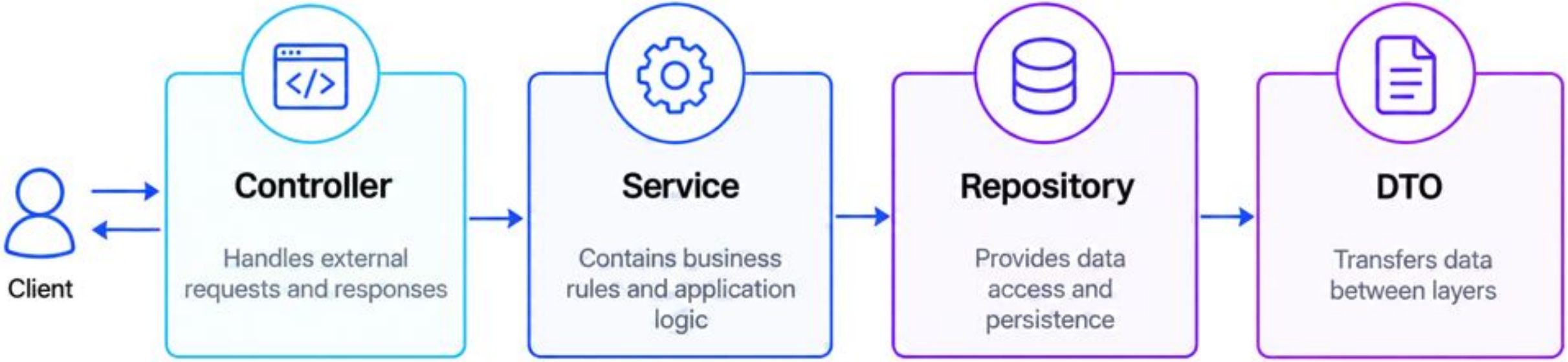
Где логика? Кто от кого зависит?
За что отвечает каждый модуль?

Архитектура

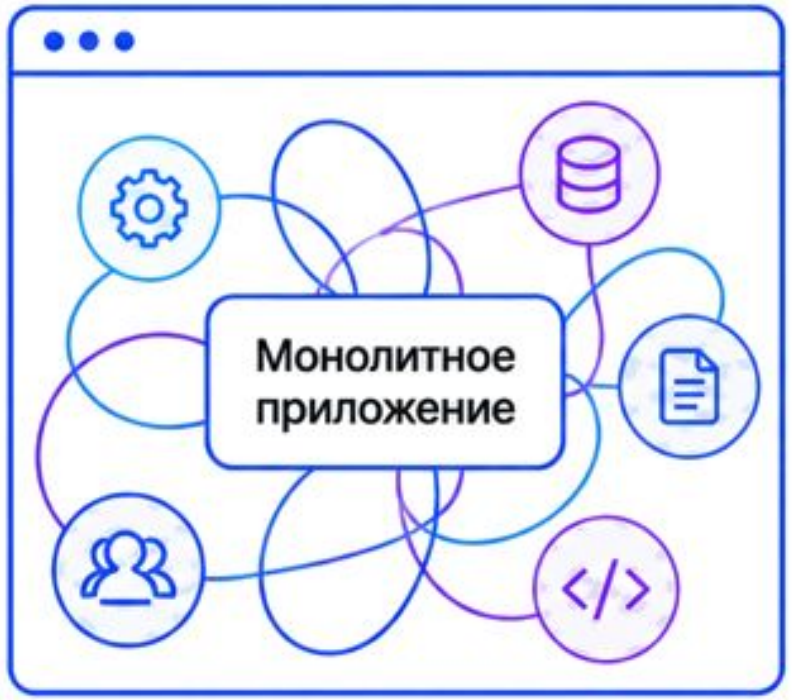
Видим роли, ответственность и зависимости

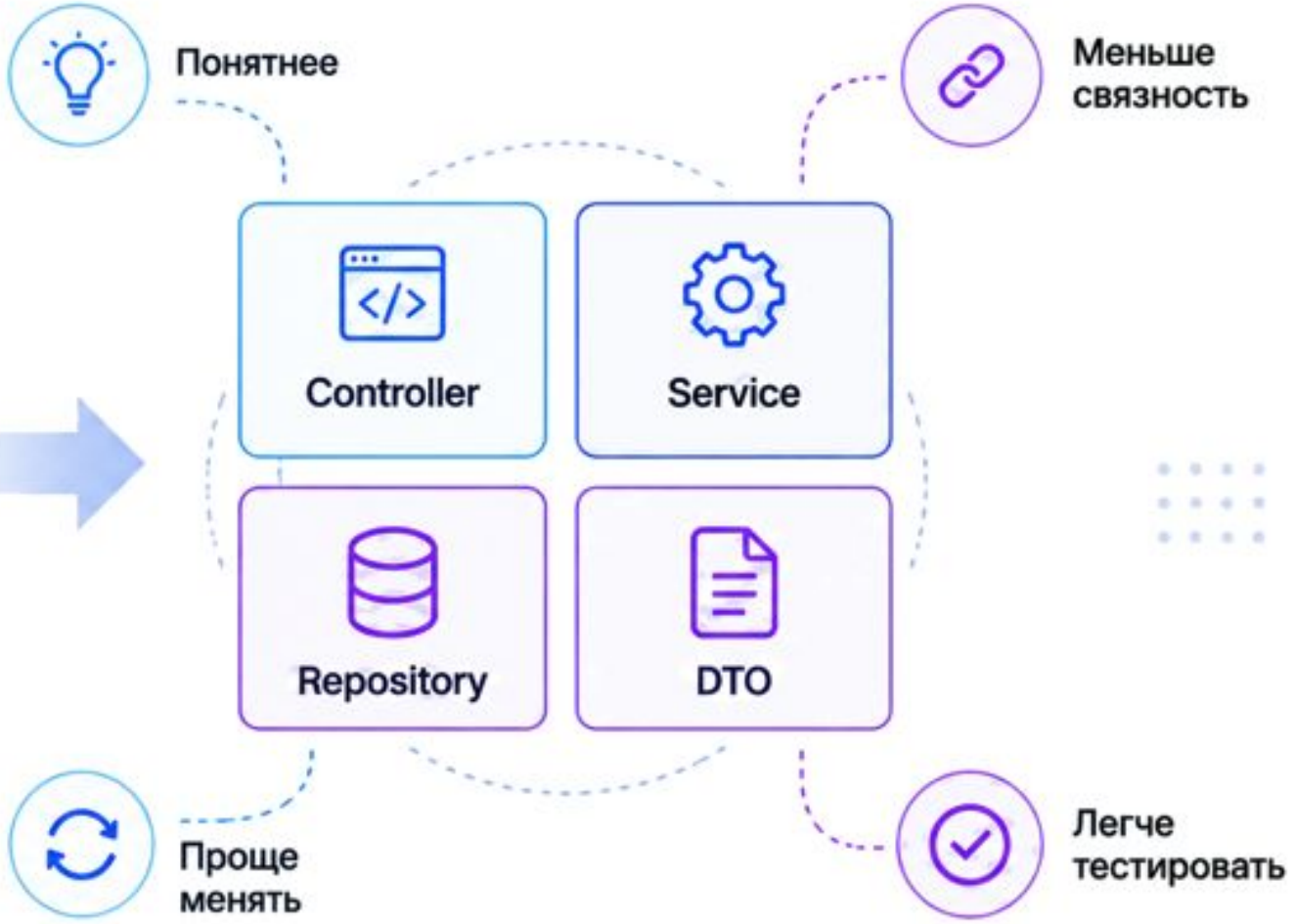


Separation of Concerns

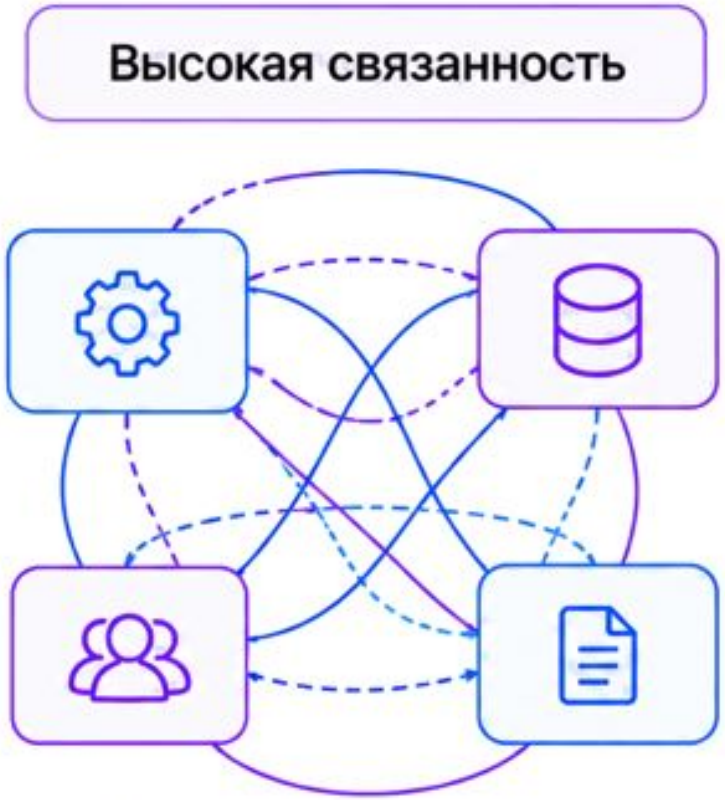


Зачем разделять ответственности

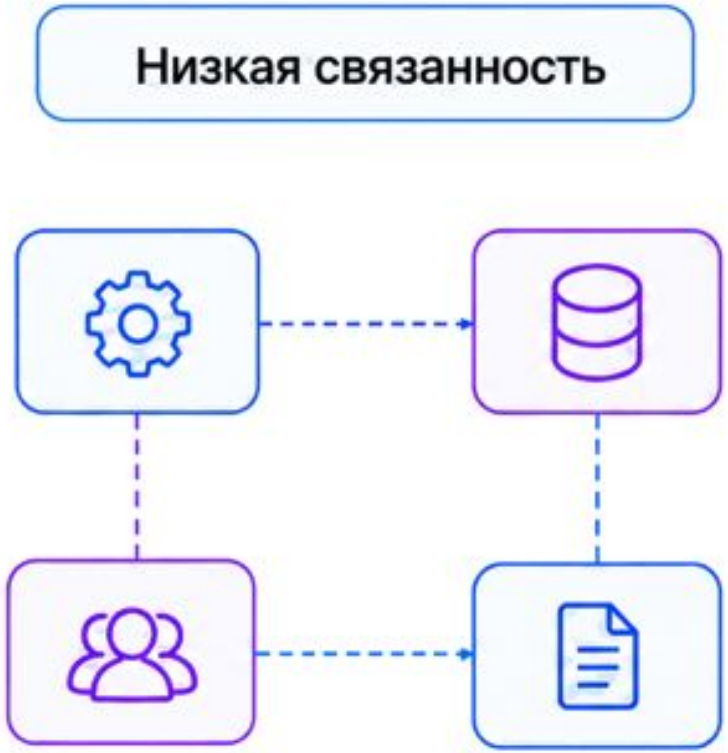




Coupling

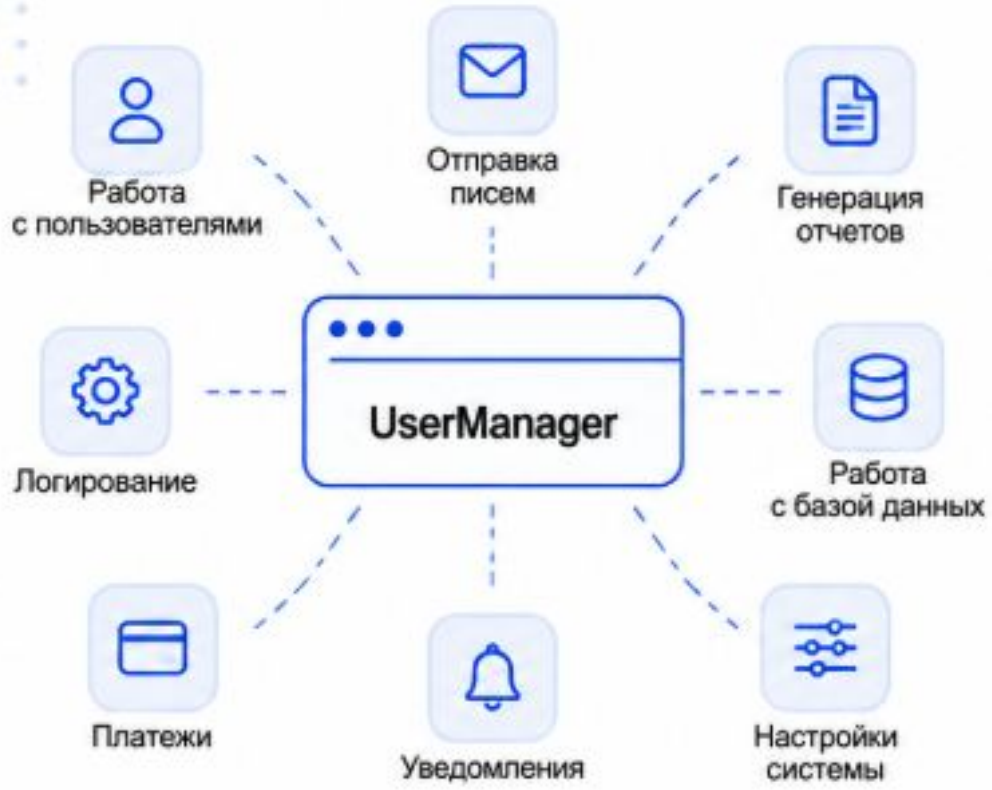


Сложнее изменять



Проще изменять

Cohesion



Низкая связность (Low Cohesion)

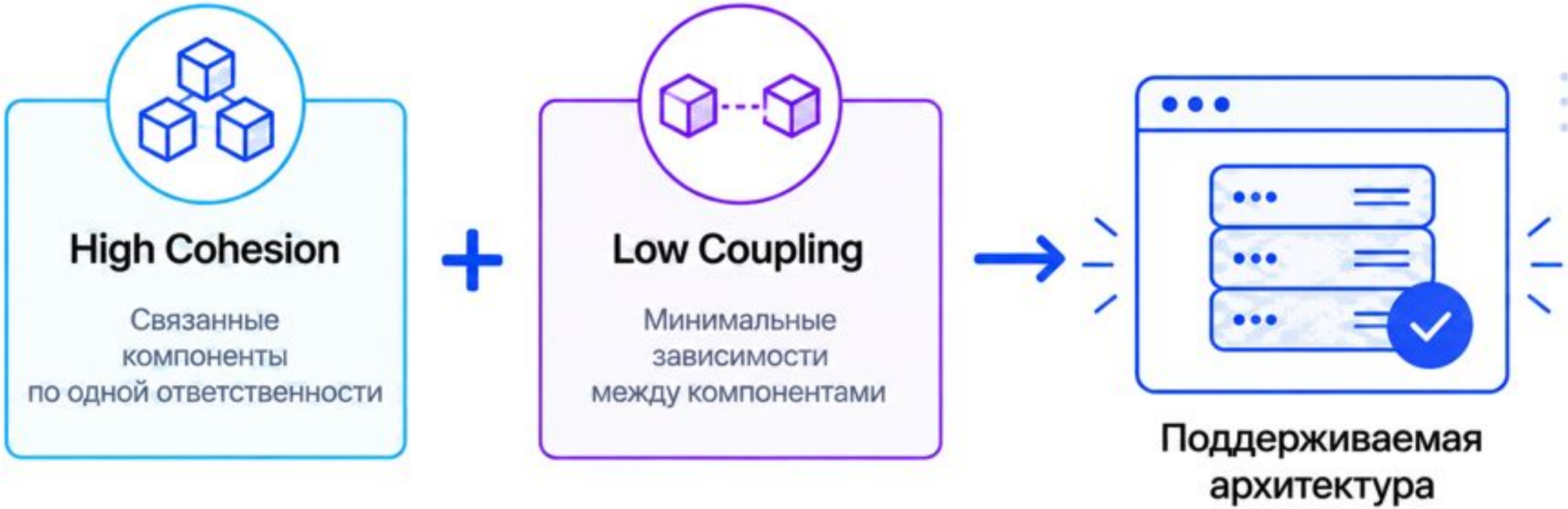
Один компонент отвечает за множество разных задач.



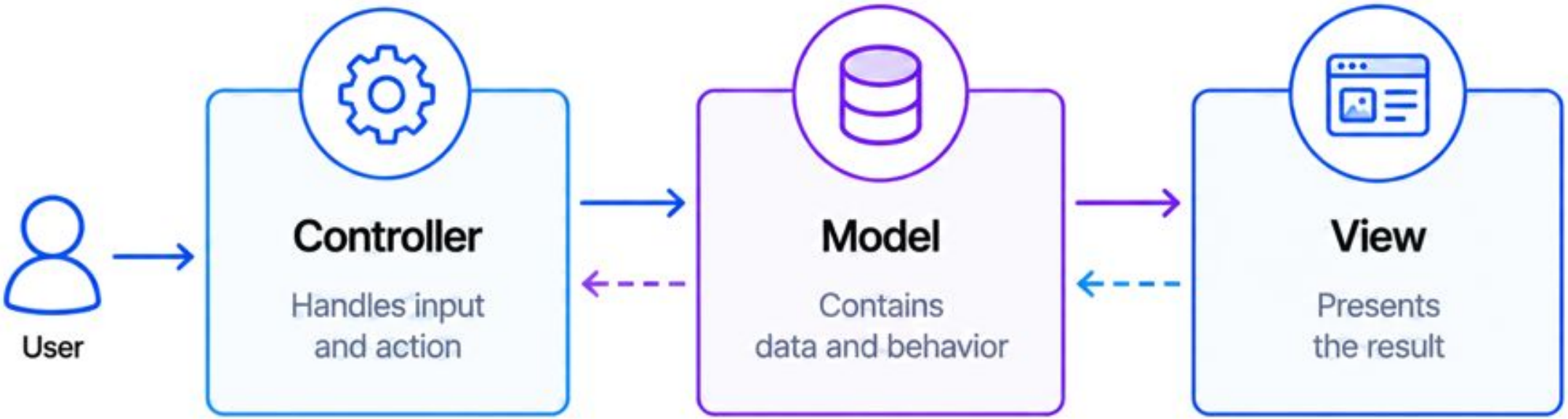
Высокая связность (High Cohesion)

Компонент отвечает за одну конкретную область ответственности.

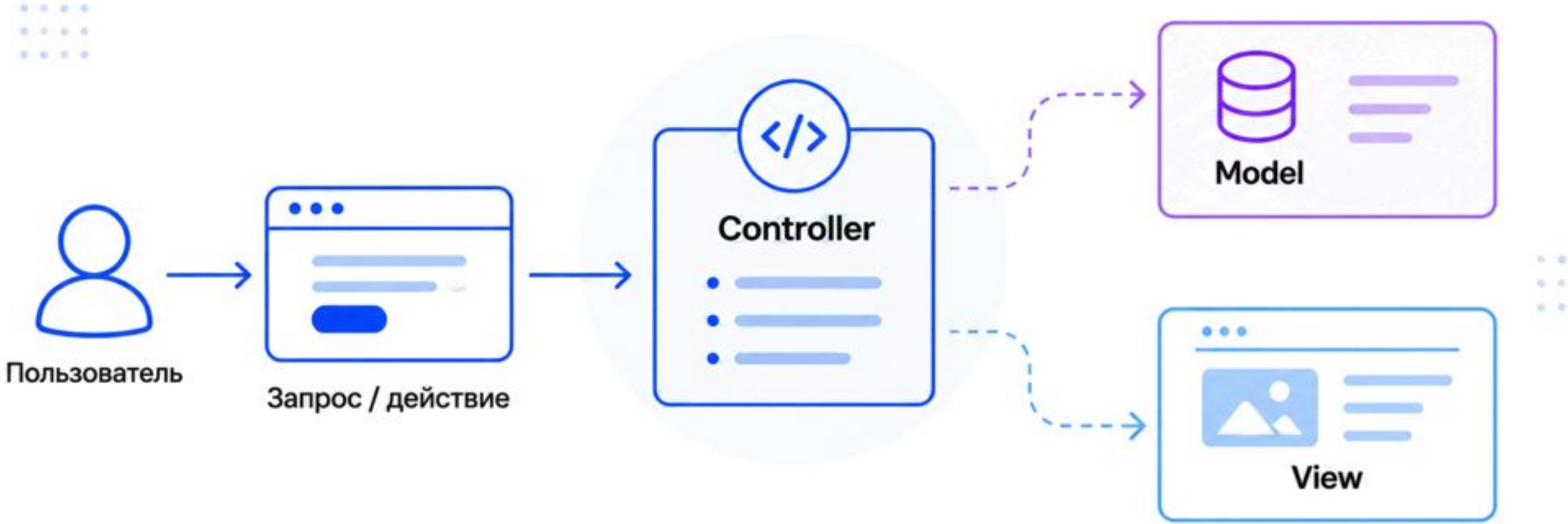
Хорошая архитектурная цель



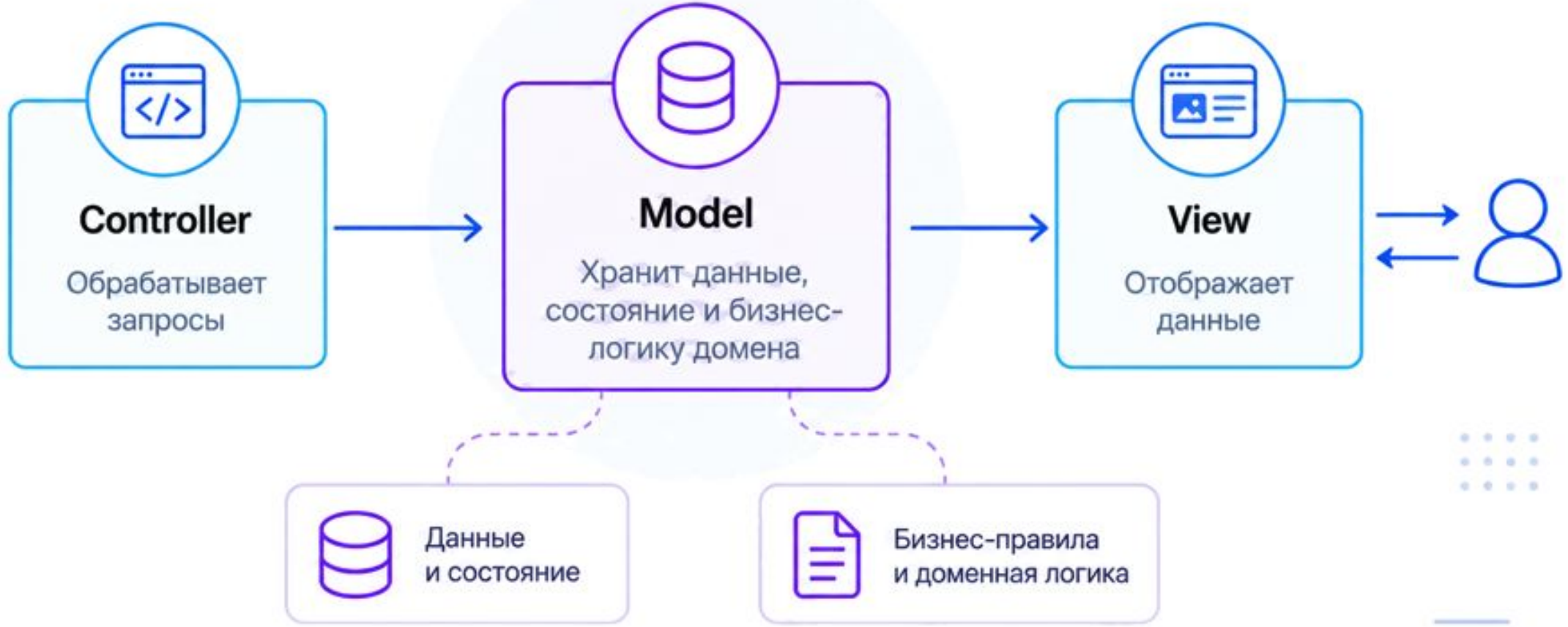
MVC


User
Controller
Handles input
and action
Model
Contains
data and behavior
View
Presents
the result

Controller в MVC



Model в MVC


Controller
Обрабатывает запросы
Model
Хранит данные, состояние и бизнес-логику домена
View
Отображает данные
Данные и состояние
Бизнес-правила и доменная логика

View в MVC



Model

Данные приложения

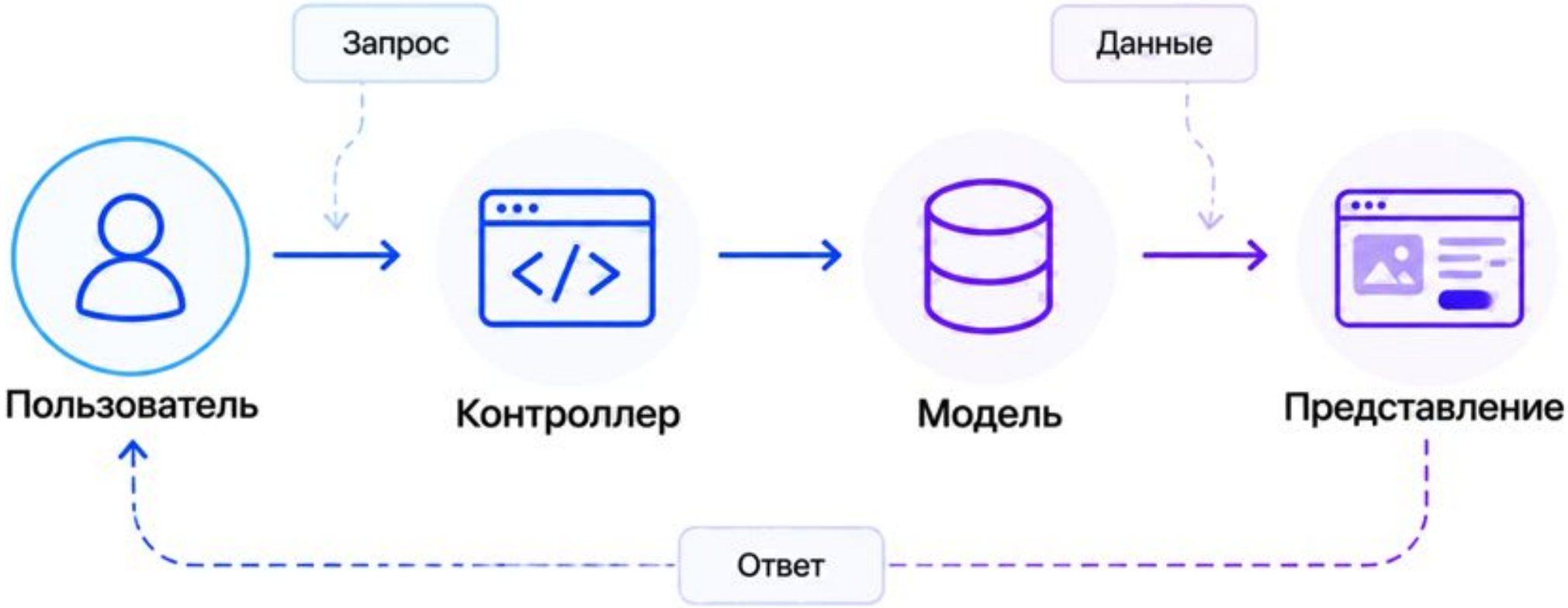
View

Представление данных пользователю

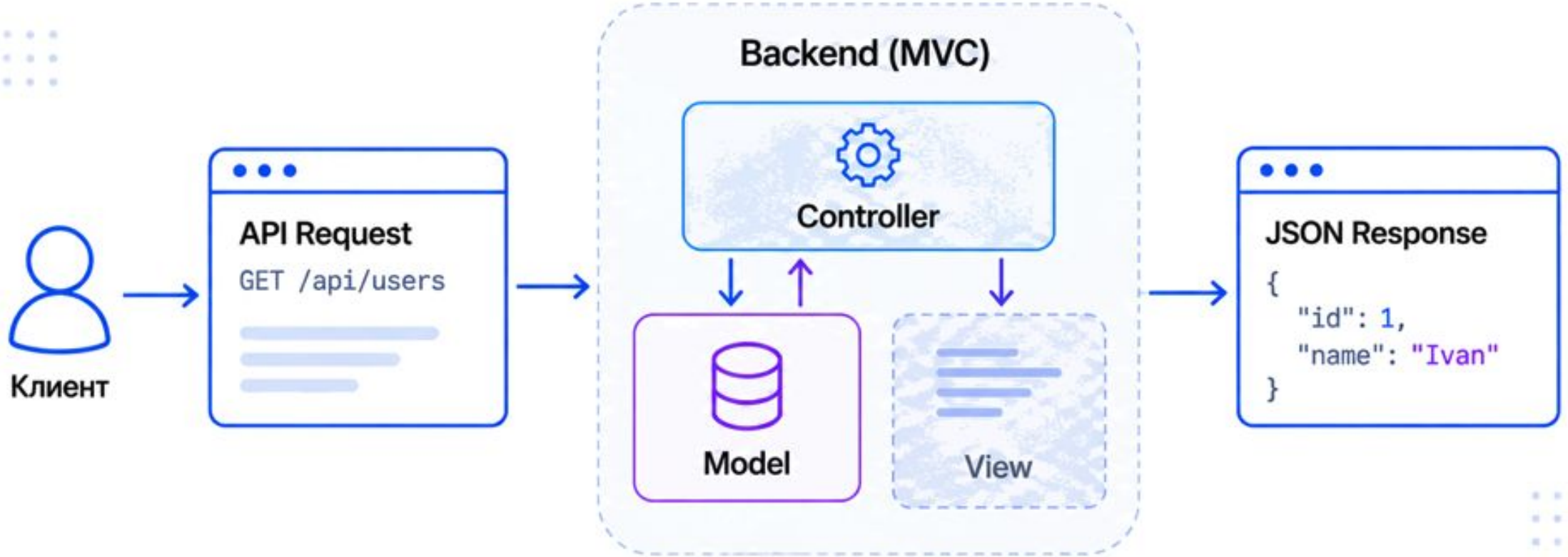
Пользователь

Видит результат и взаимодействует

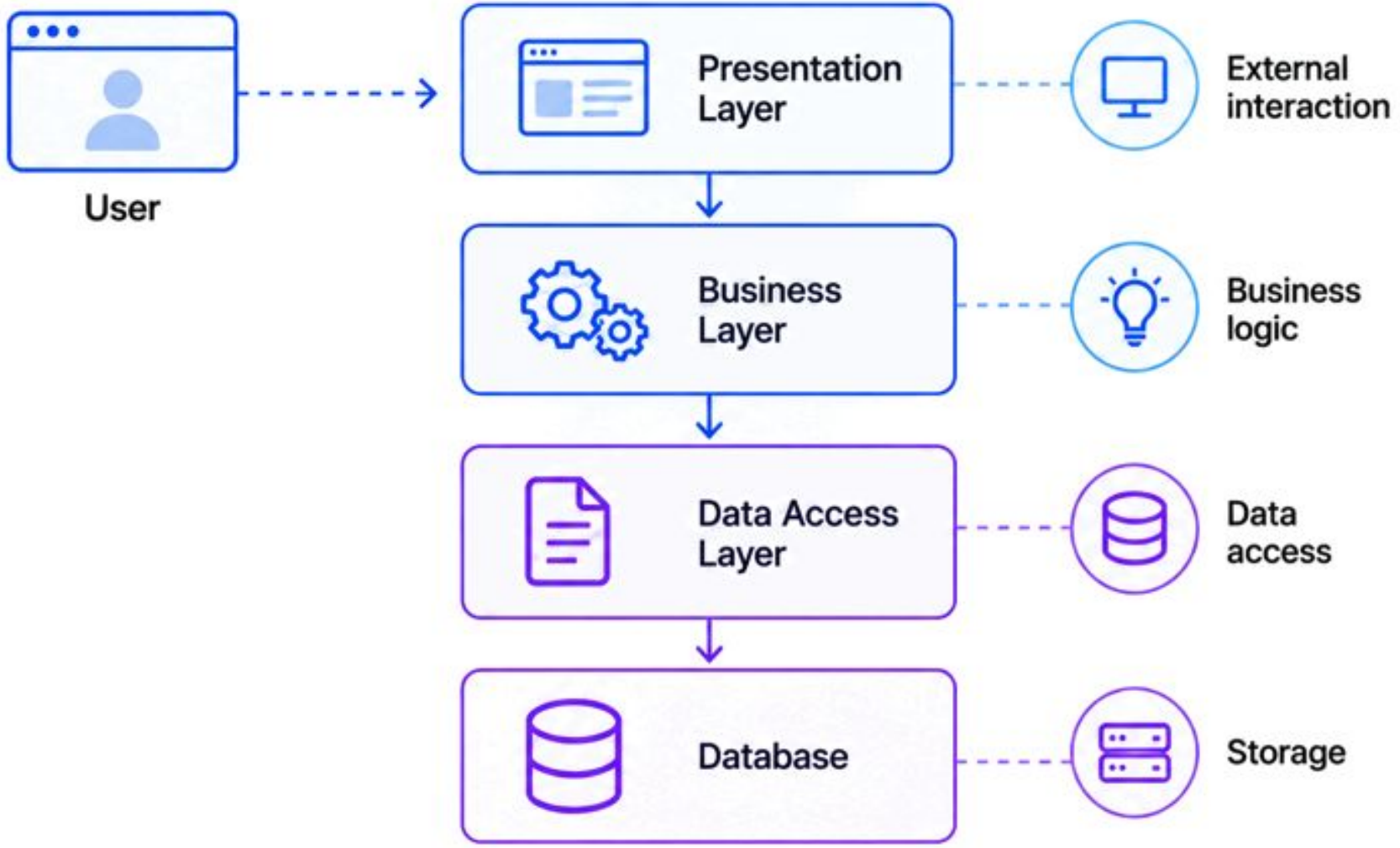
Классический MVC



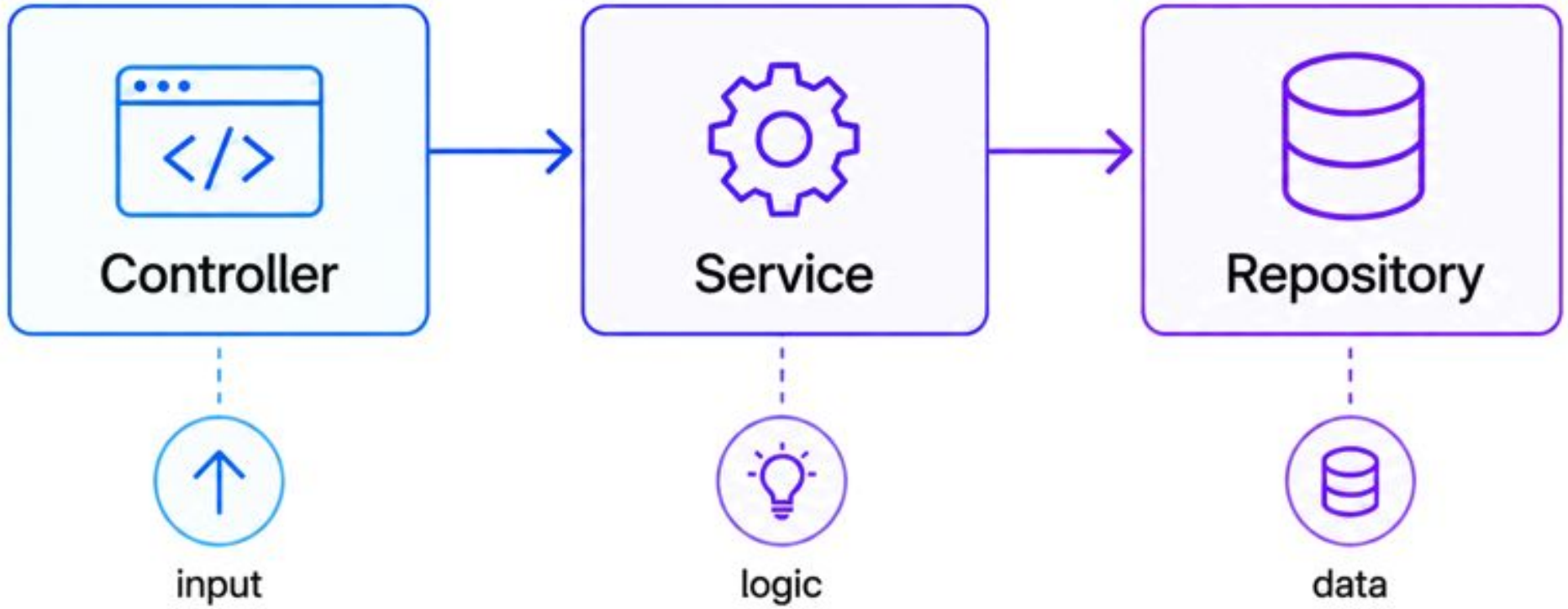
MVC в backend-приложениях



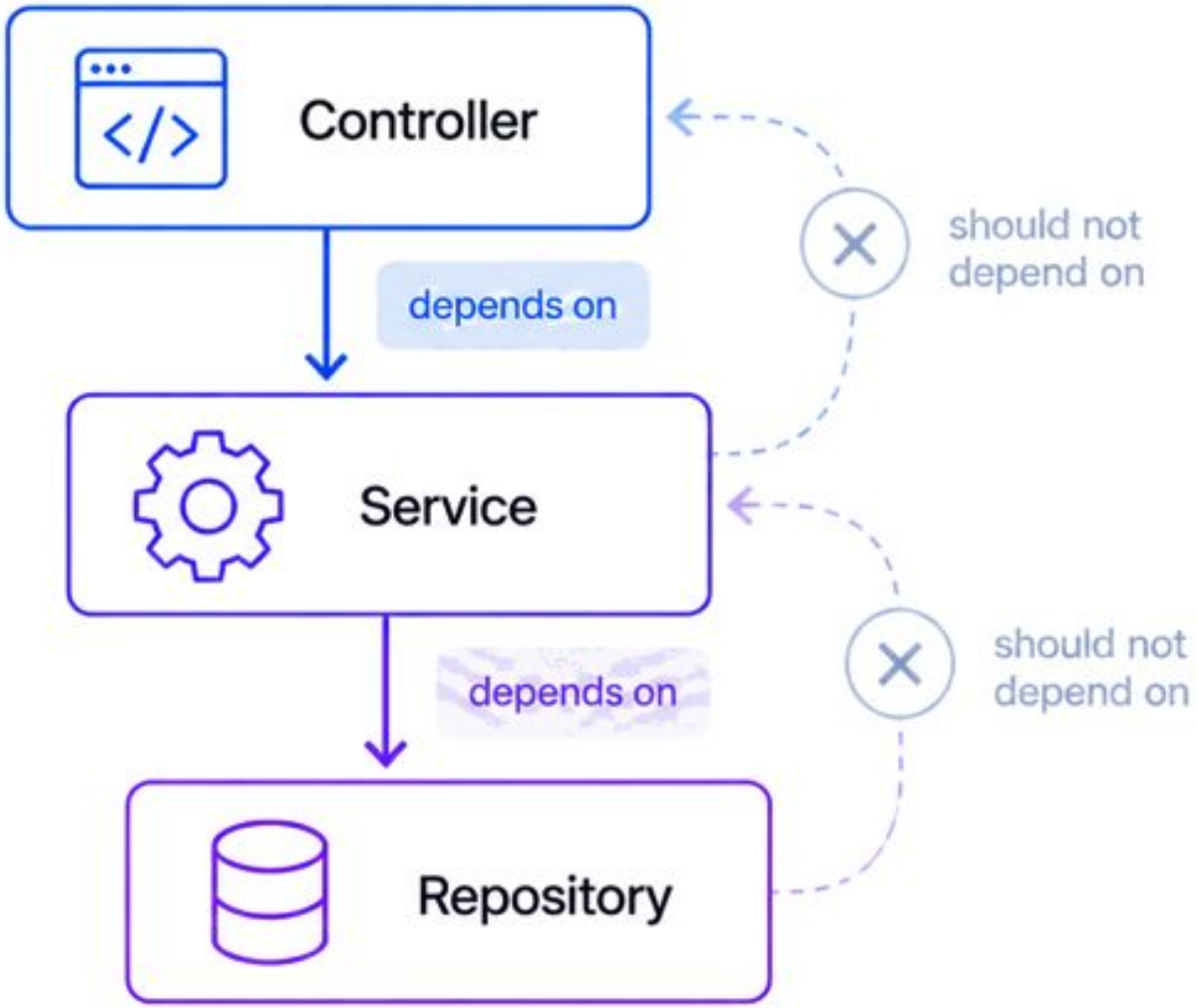
Layered Architecture



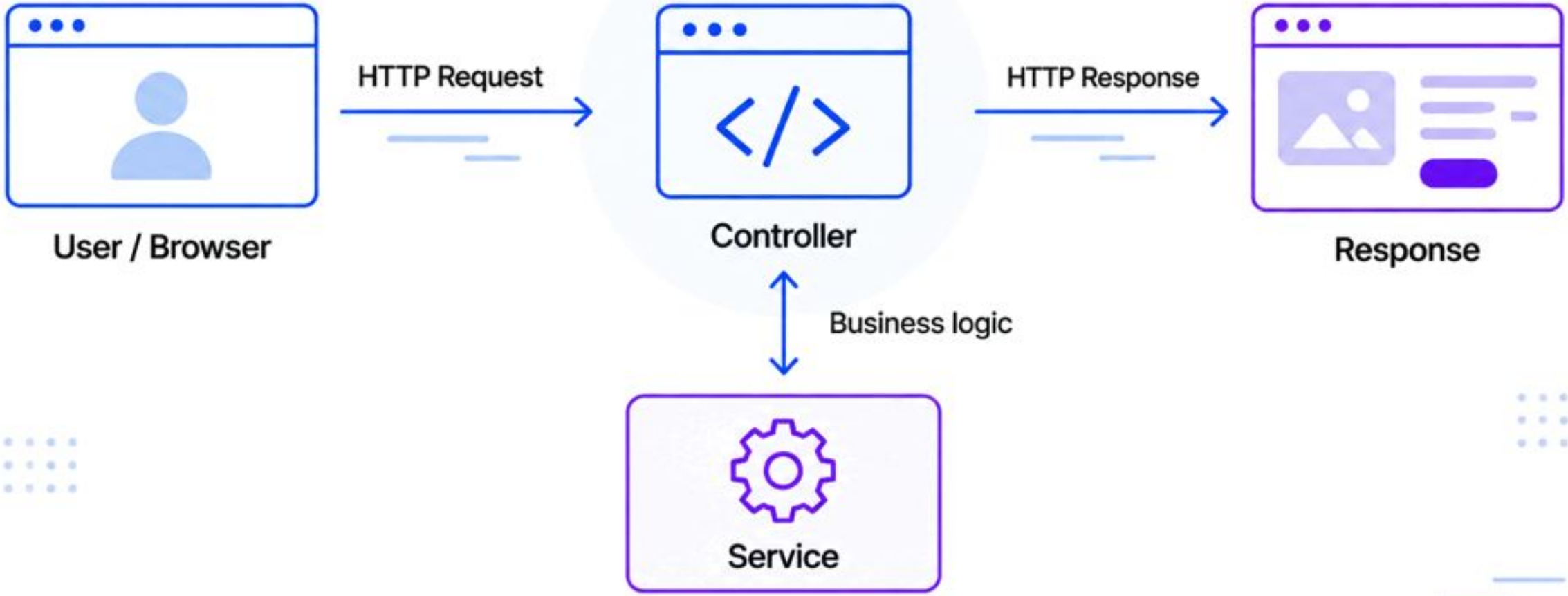
Controller / Service / Repository



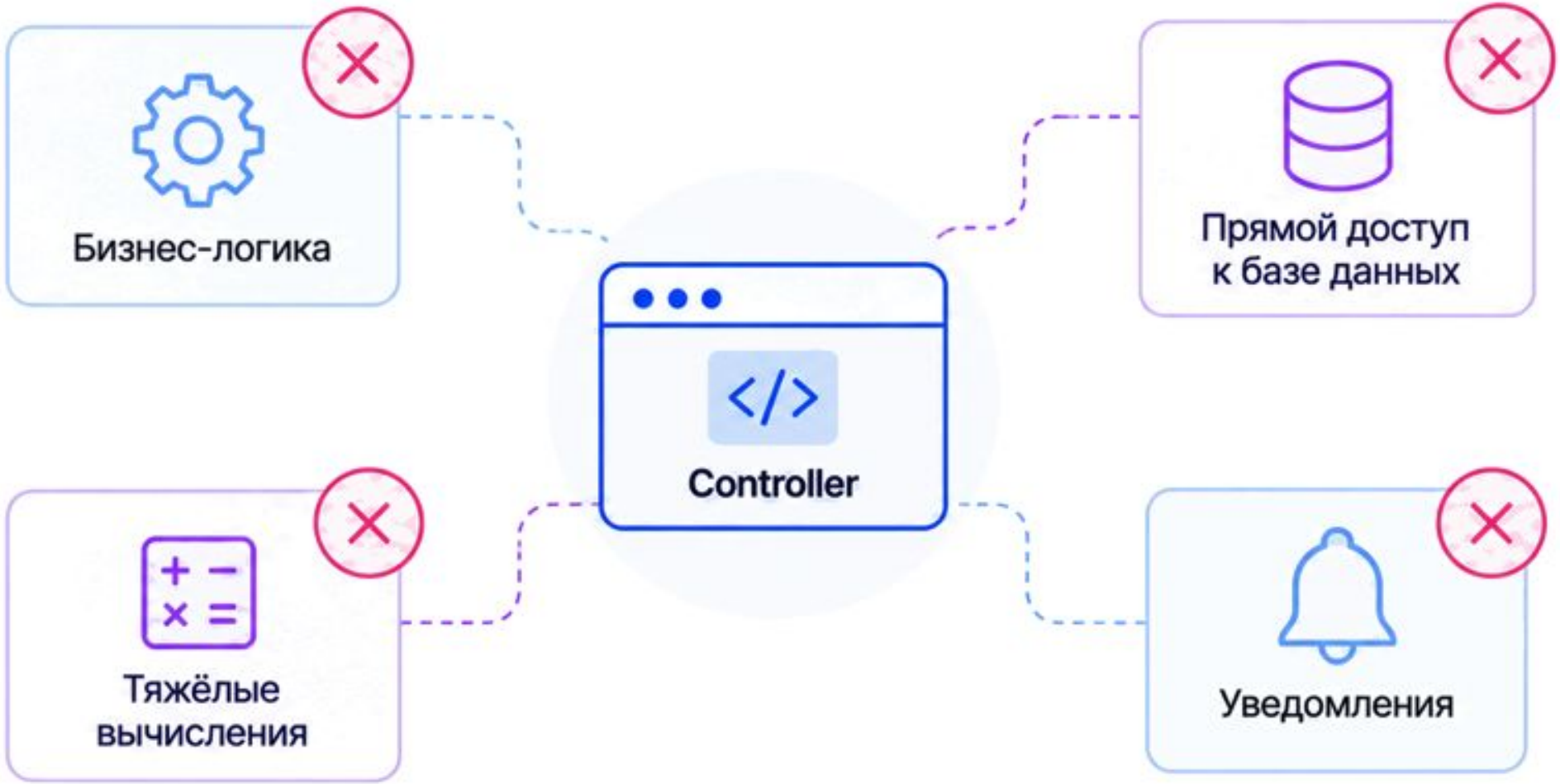
Направление зависимостей



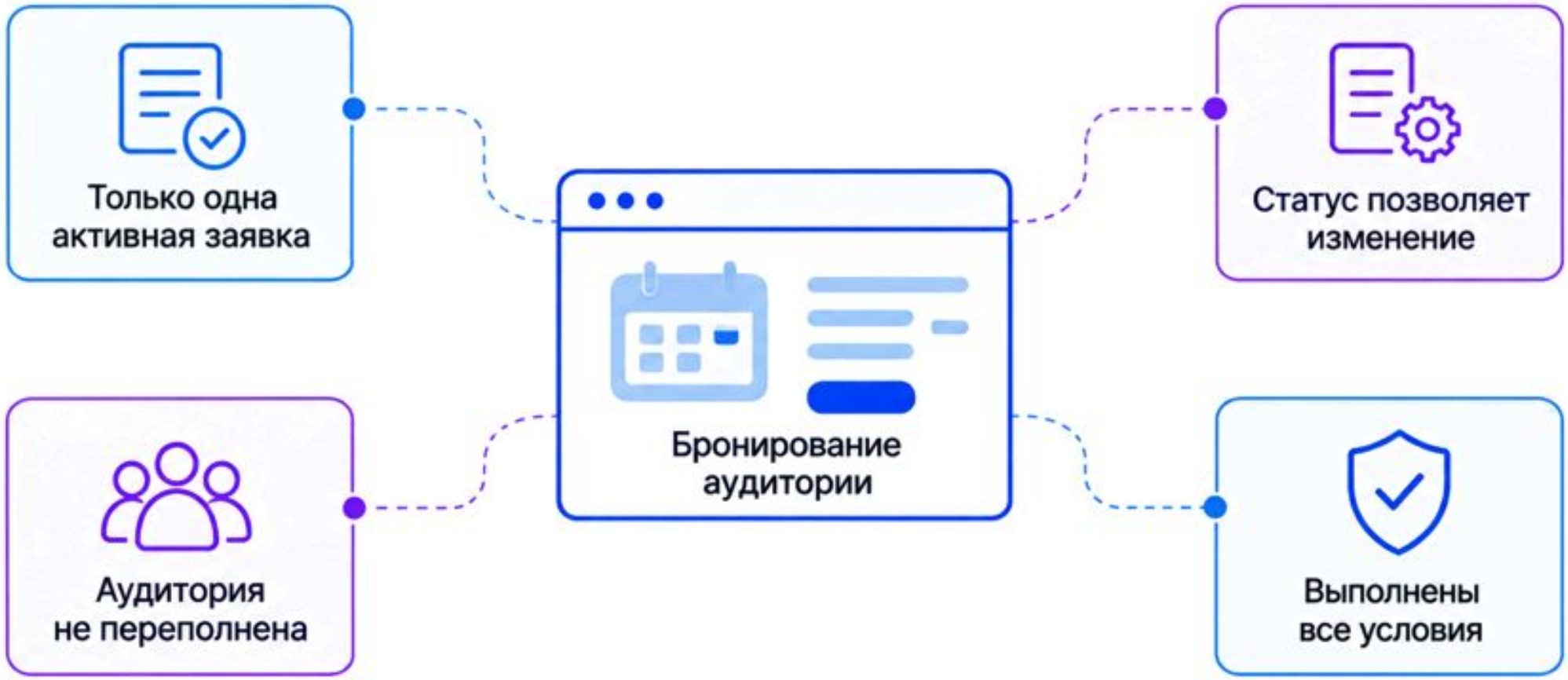
Controller Layer



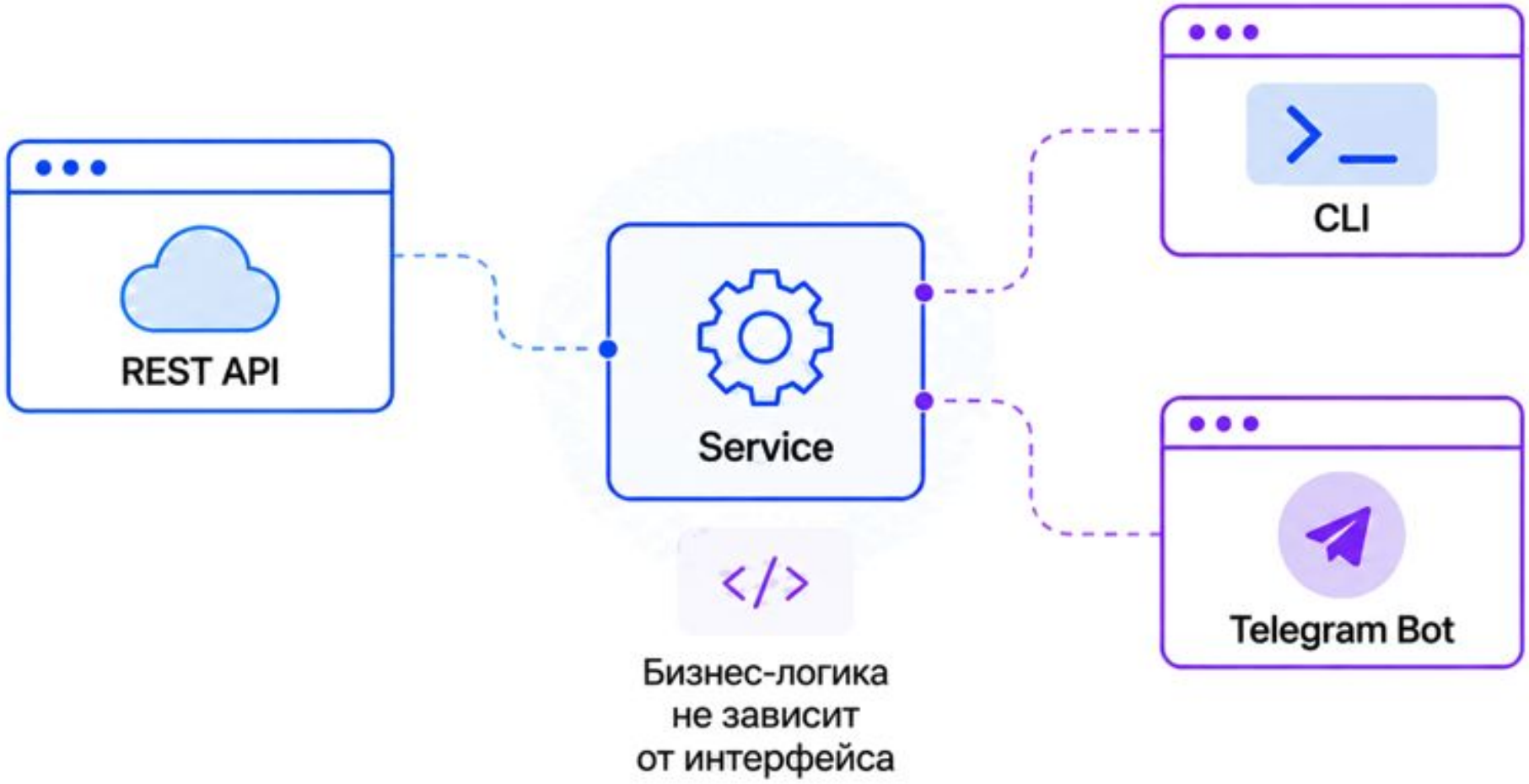
Чего не должен делать Controller



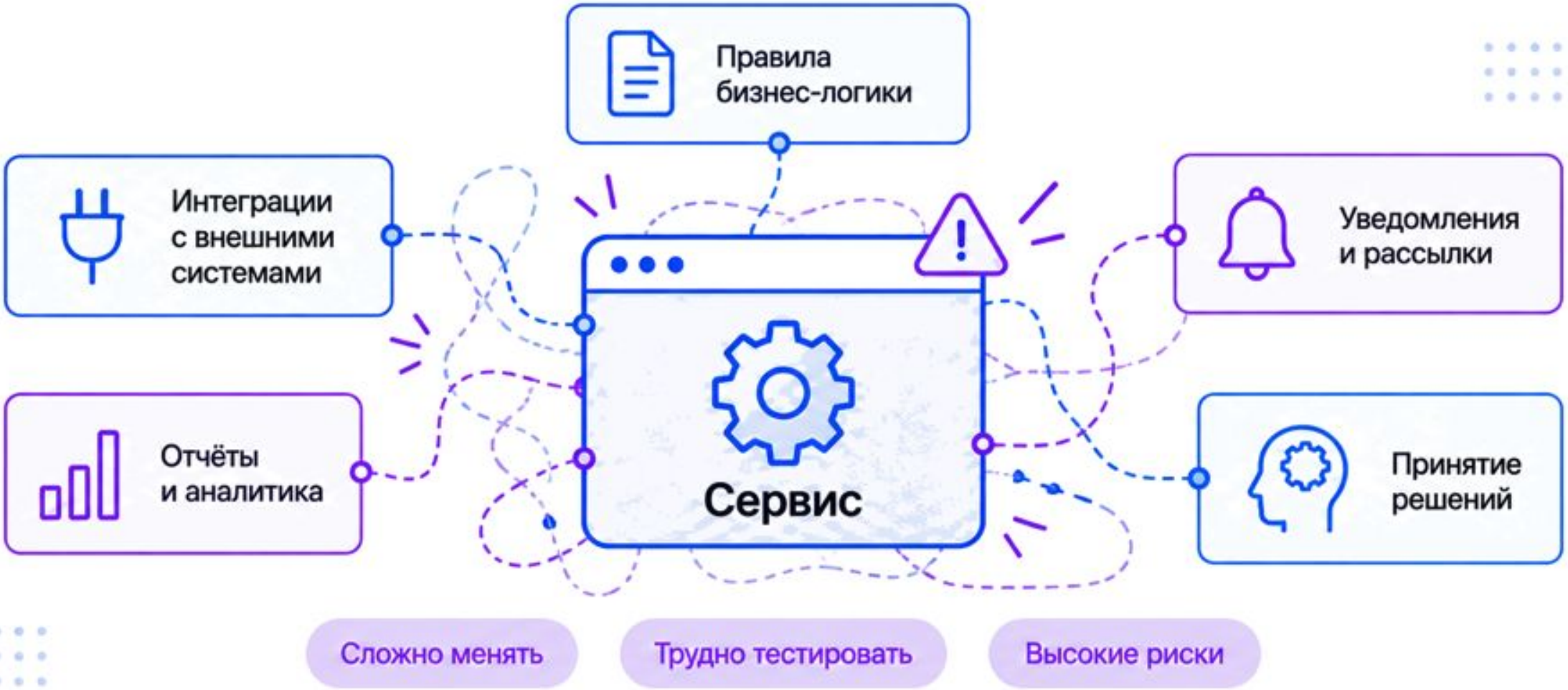
Business Logic



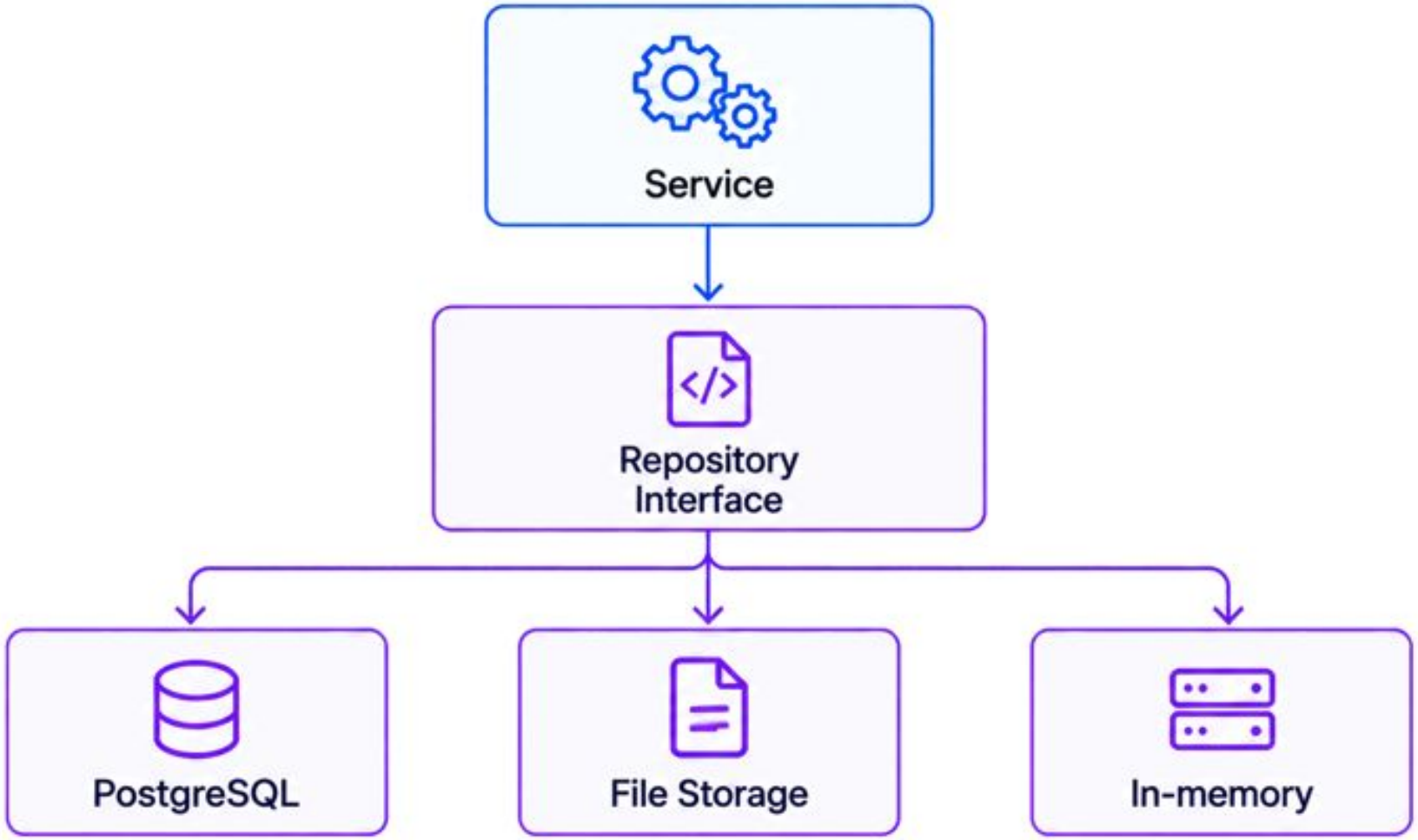
Service не зависит от интерфейса



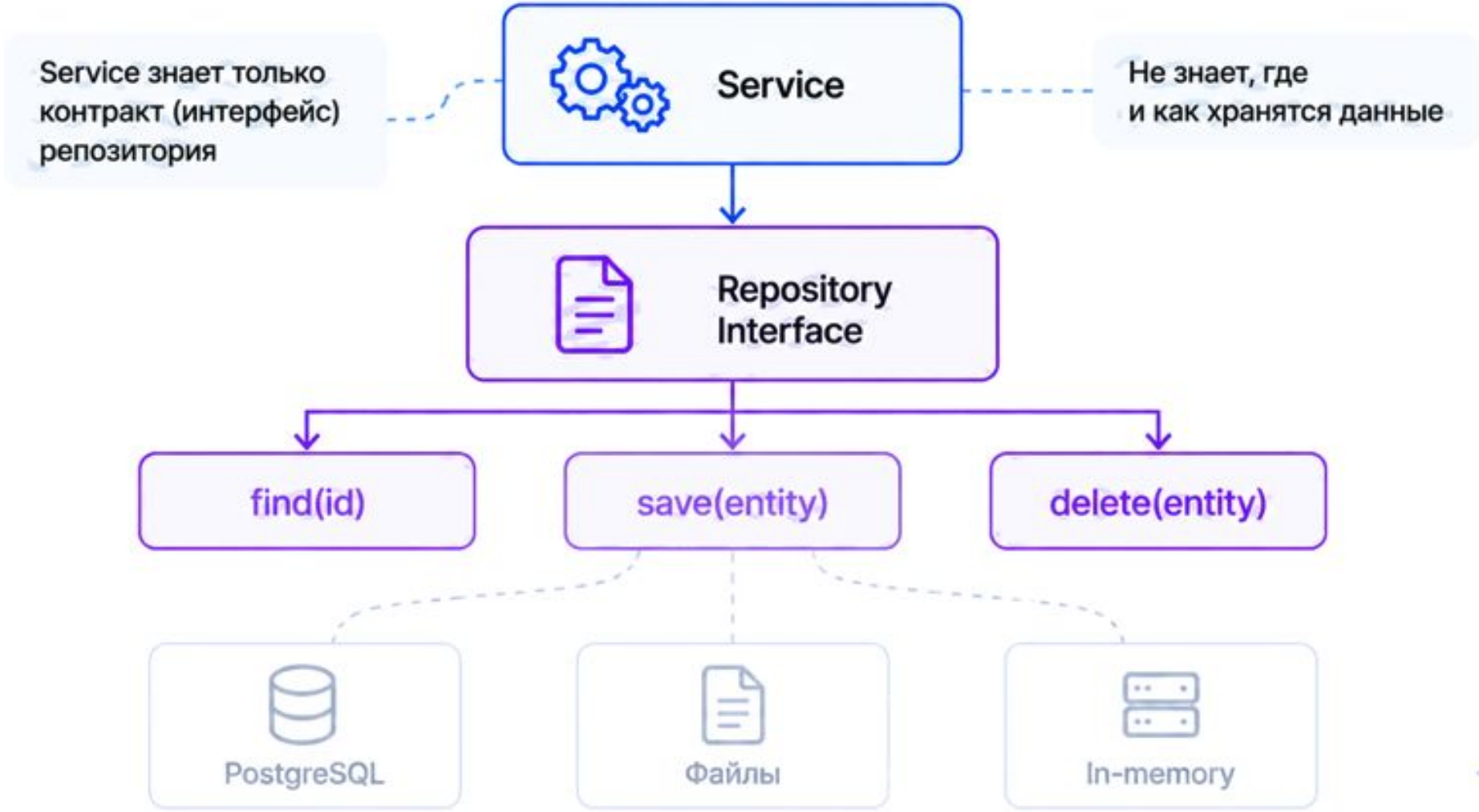
God Servise


Правила
бизнес-логики
Интеграции
с внешними
системами
Уведомления
и рассылки
Отчёты
и аналитика
Принятие
решений
Сервис
Сложно менять
Трудно тестировать
Высокие риски

Repository как абстракция



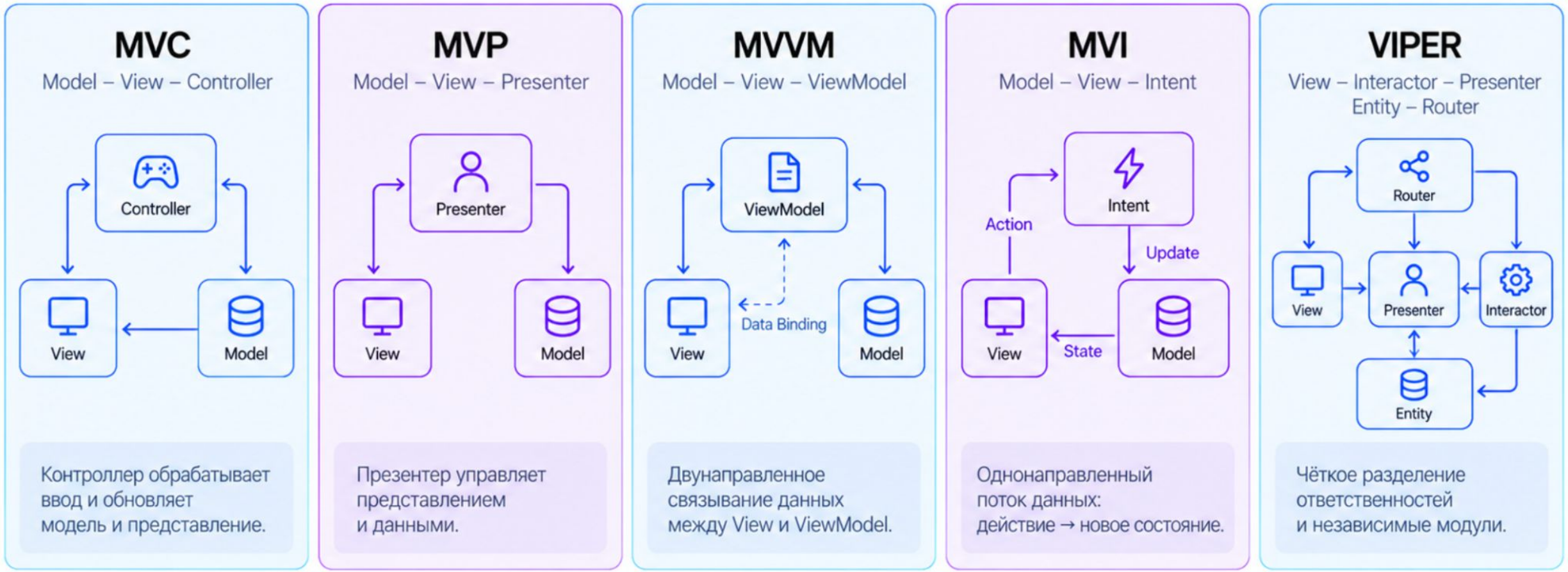
Что видит Service



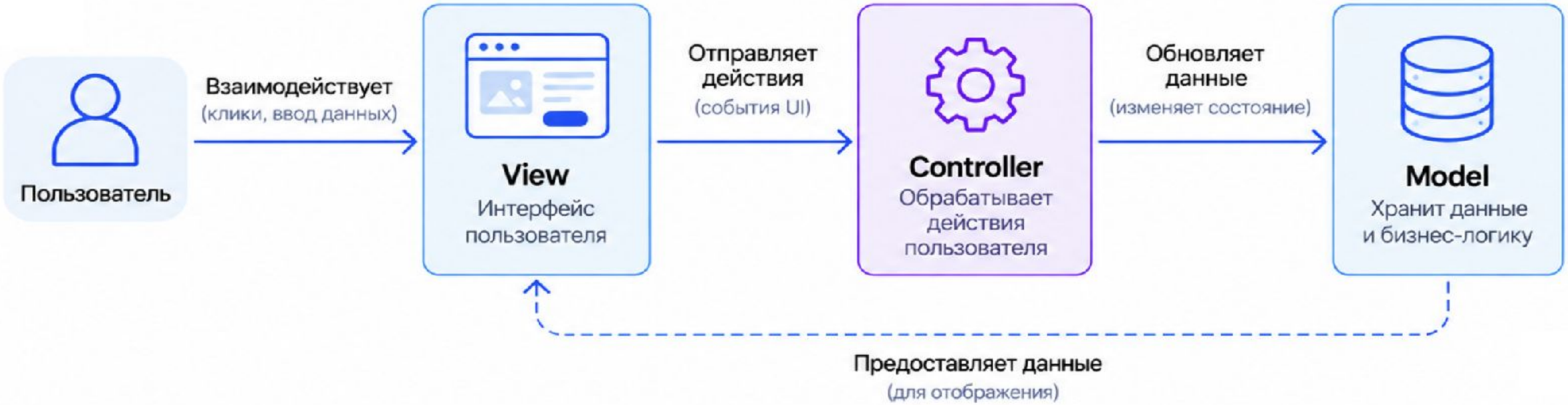
Зачем нужен Repository



Архитектуры UI: MVC, MVP, MVVM, MVI, VIPER



MVC



View
Показывает интерфейс

Отображает данные модели и принимает действия пользователя.

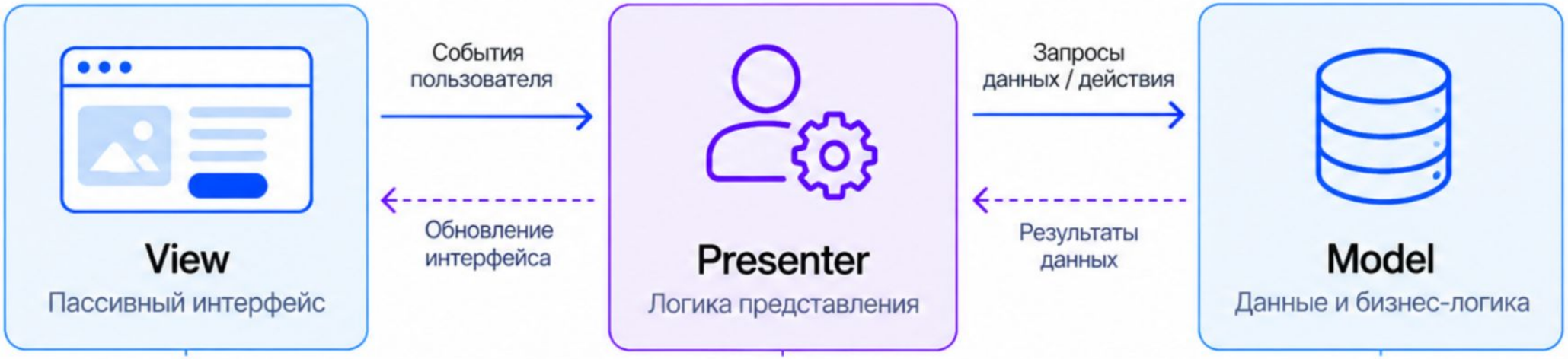
Controller
Обрабатывает действия

Получает действия из View, вызывает изменения в модели и управляет логикой приложения.

Model
Хранит данные

Содержит данные и бизнес-логику приложения, не зависит от интерфейса.

MVP



View — пассивный компонент

Отображает состояние, переданное Presenter, и передаёт ему события пользователя.

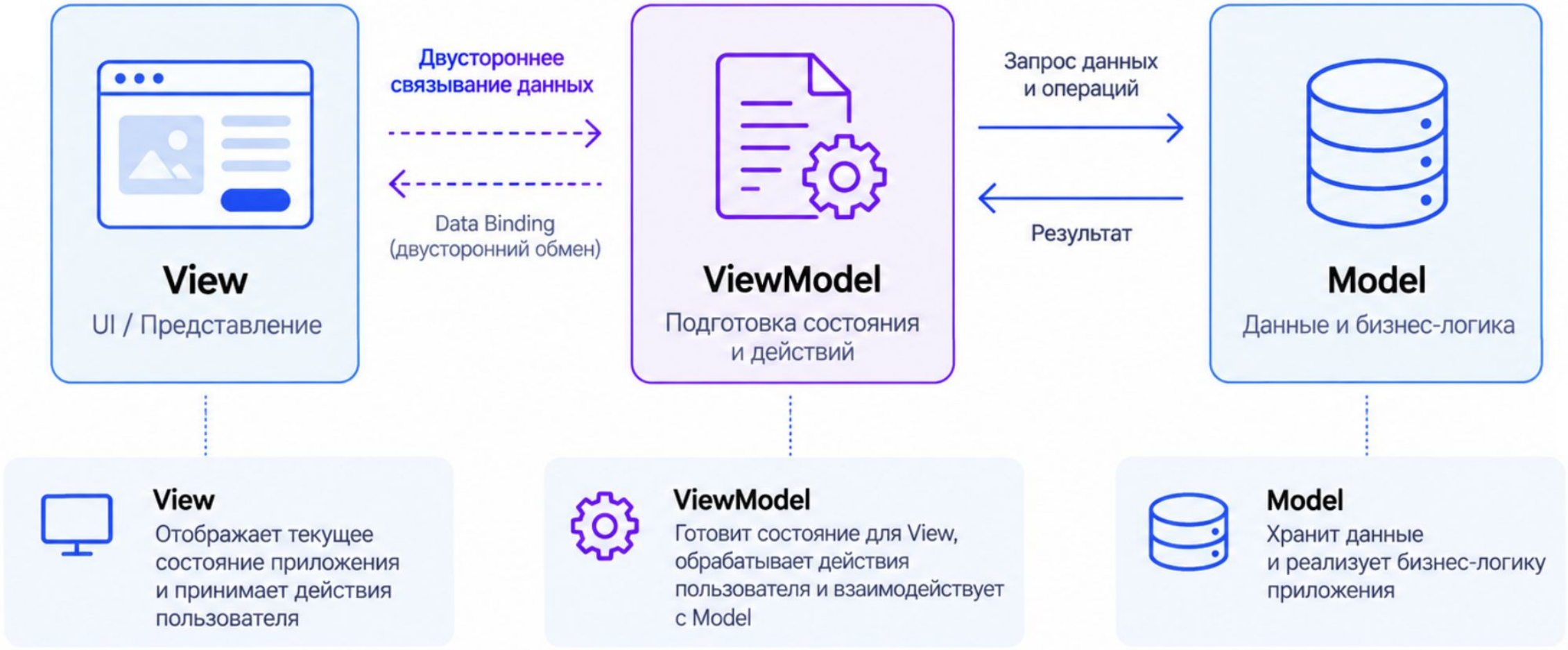
Presenter — обрабатывает логику интерфейса

Реагирует на события View, запрашивает и обрабатывает данные из Model, подготавливает данные для отображения и обновляет View.

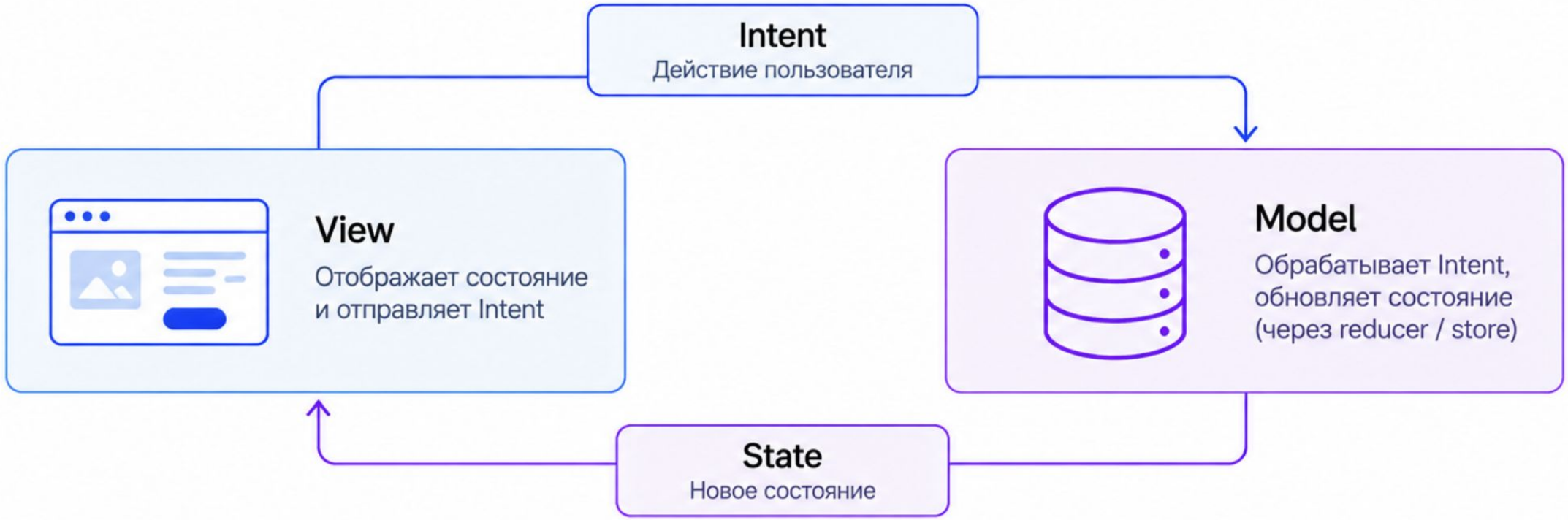
Model — данные и бизнес-логика

Содержит данные приложения и бизнес-правила. Не зависит от интерфейса и ничего не знает о View и Presenter.

MVVC


View
UI / Представление
Двустороннее связывание данных
Data Binding (двусторонний обмен)
ViewModel
Подготовка состояния и действий
Запрос данных и операций
Результат
Model
Данные и бизнес-логика
View
Отображает текущее состояние приложения и принимает действия пользователя
ViewModel
Готовит состояние для View, обрабатывает действия пользователя и взаимодействует с Model
Model
Хранит данные и реализует бизнес-логику приложения

MVI



View

Показывает UI на основе состояния и отправляет пользовательские действия в виде Intent.

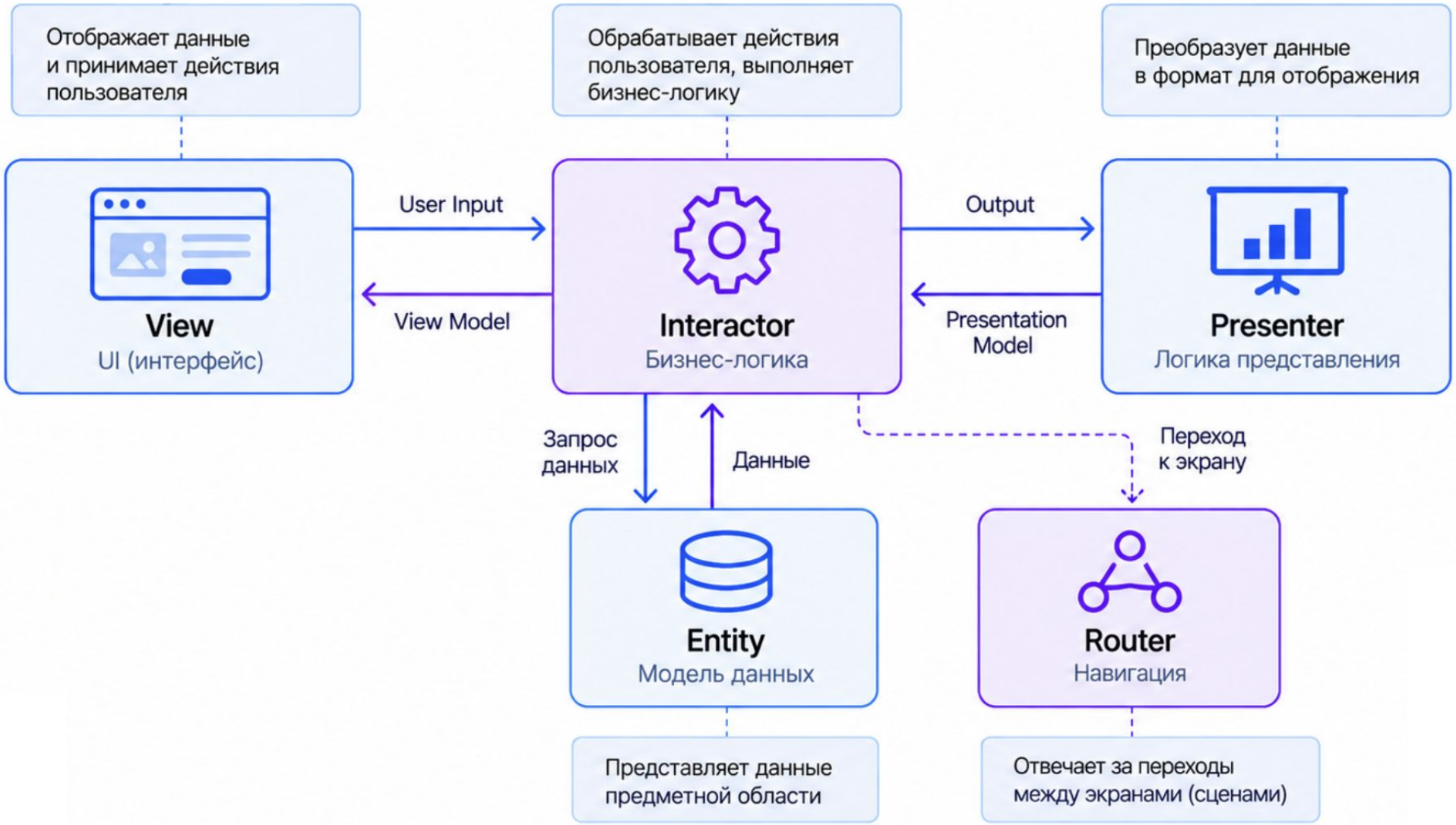
Model (Reducer / Store)

Принимает Intent, выполняет бизнес-логику и создаёт новое состояние.

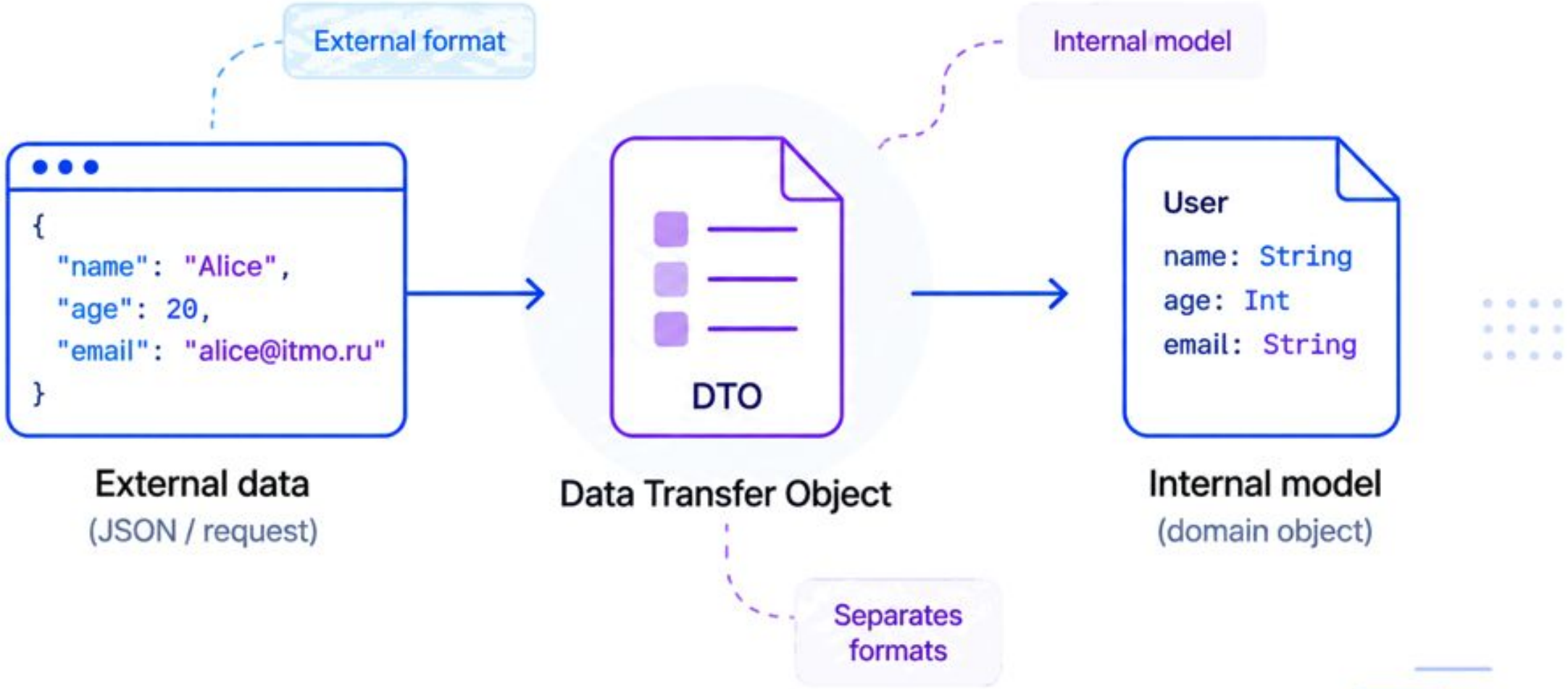
Single Source of Truth

Весь UI строится на одном источнике истины — состоянии, которое хранится в модели.

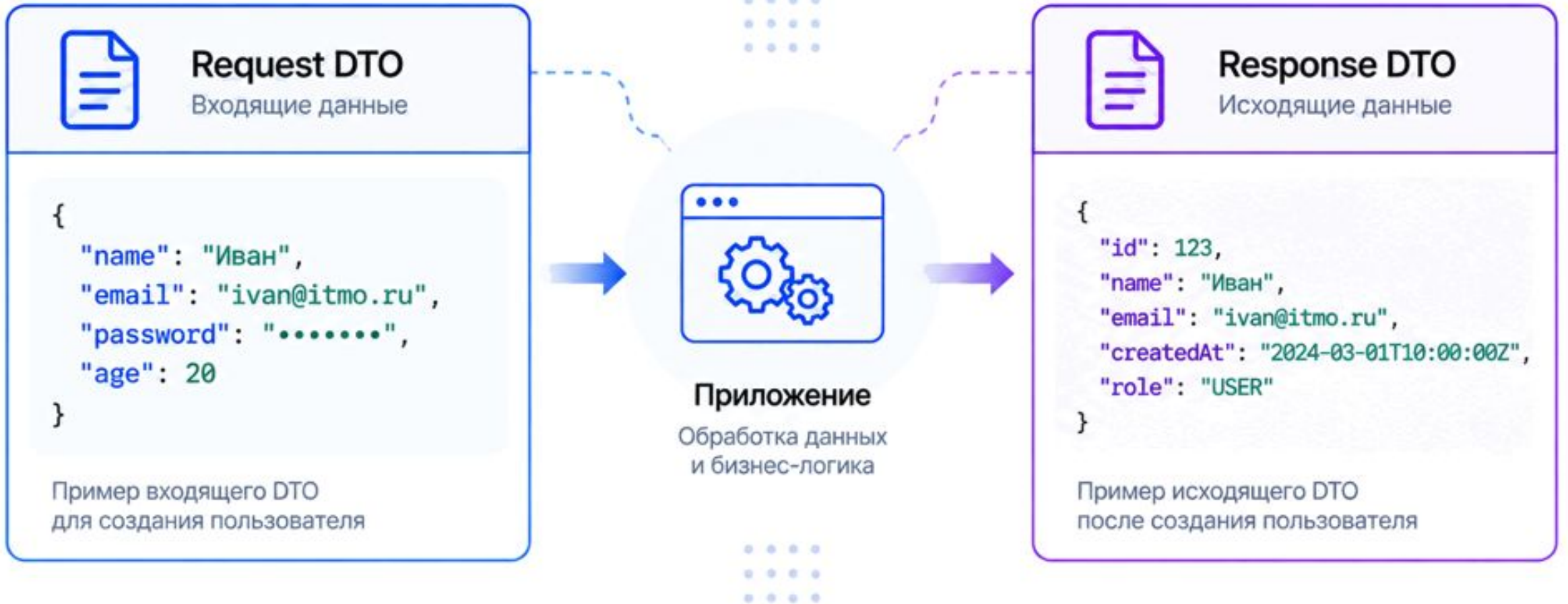
VIPER


Отображает данные
и принимает действия
пользователя
Обрабатывает действия
пользователя, выполняет
бизнес-логику
Преобразует данные
в формат для отображения
View
UI (интерфейс)
User Input
View Model
Interactor
Бизнес-логика
Output
Presentation
Model
Presenter
Логика представления
Запрос
данных
Данные
Переход
к экрану
Entity
Модель данных
Router
Навигация
Представляет данные
предметной области
Отвечает за переходы
между экранами (сценами)

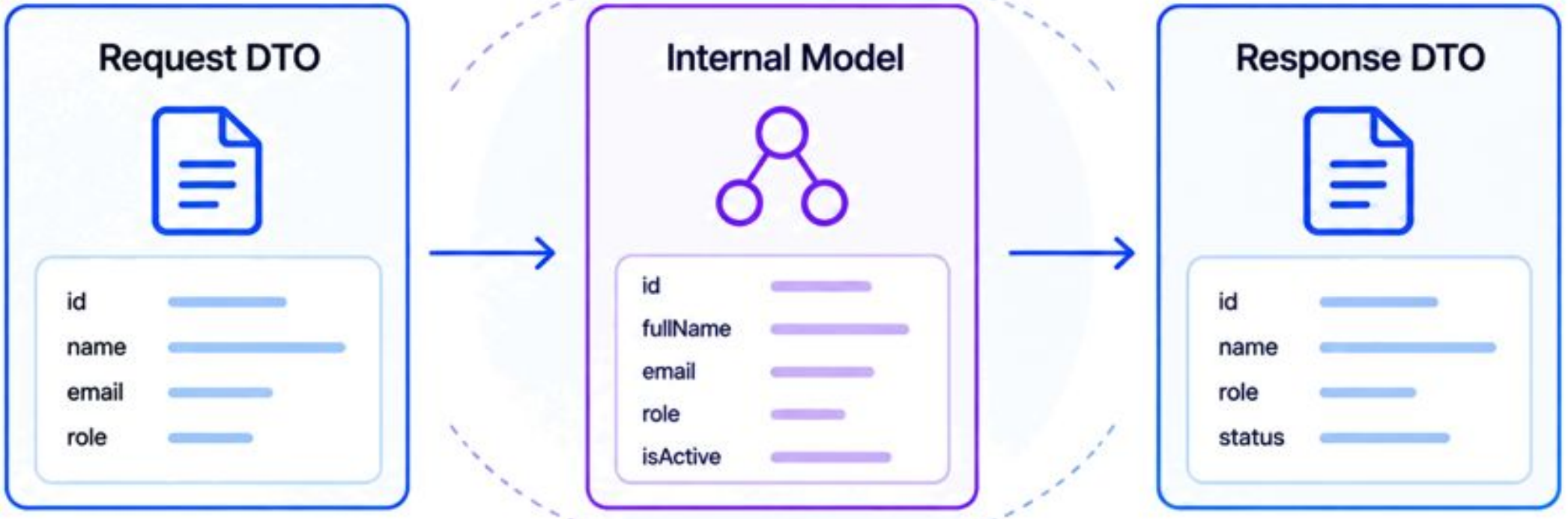
DTO — Data Transfer Object



Request DTO и Response DTO



Mapping


Request DTO
id
name
email
role
Internal Model
id
fullName
email
role
isActive
Response DTO
id
name
role
status

Validation: два типа проверки

Structural Validation

Проверка структуры и формата данных. Гарантирует корректность ввода на уровне полей.

Примеры:

Тип данных
Например, число вместо текста

Обязательное поле
Поле не должно быть пустым

Формат данных
Например, корректный email, номер телефона, дата

Business Validation

Проверка бизнес-правил и логики предметной области. Гарантирует корректность с точки зрения процессов и ограничений.

Примеры:

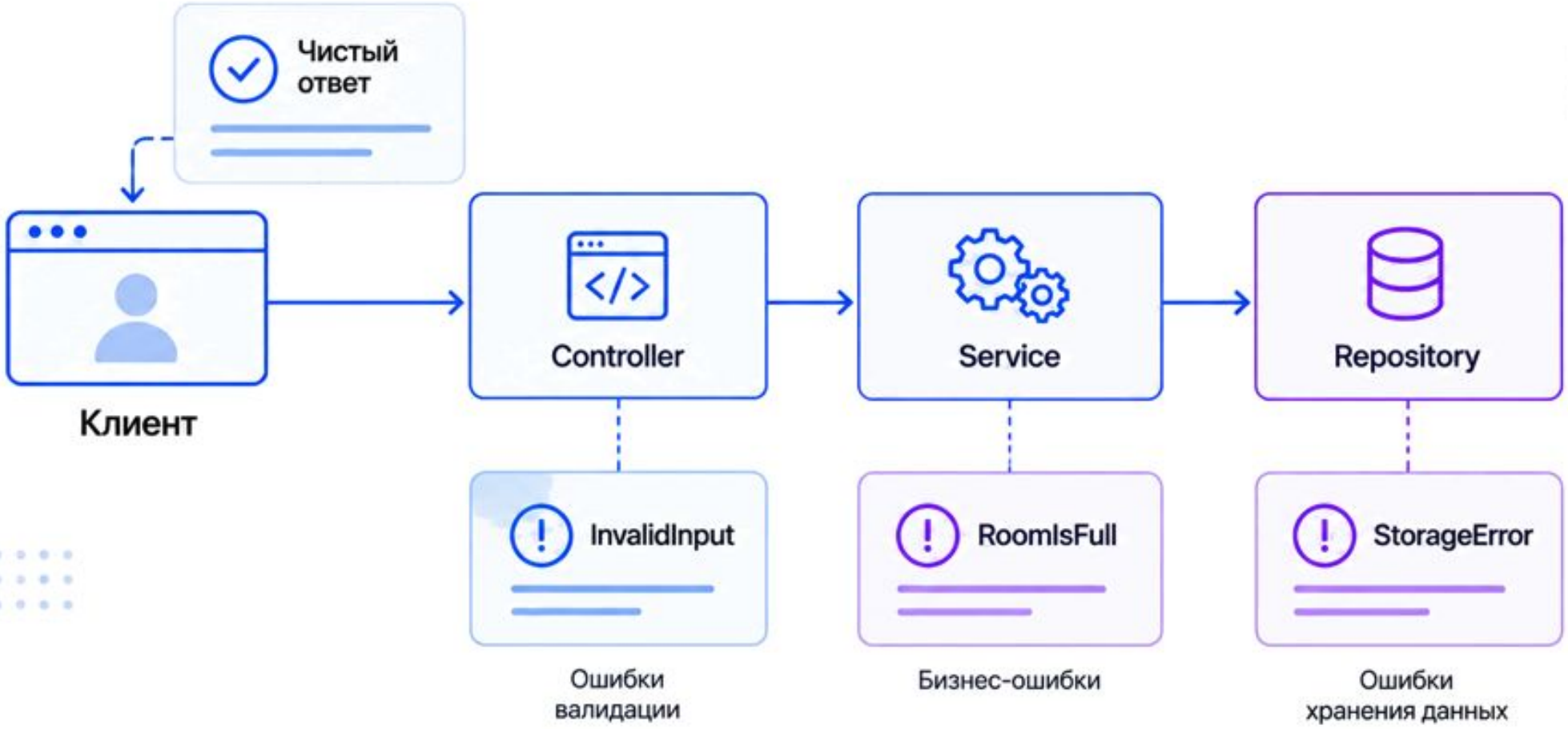
Комната уже заполнена
Нельзя забронировать, если нет мест

Ограничение по статусу
Нельзя отменить заявку в статусе Completed

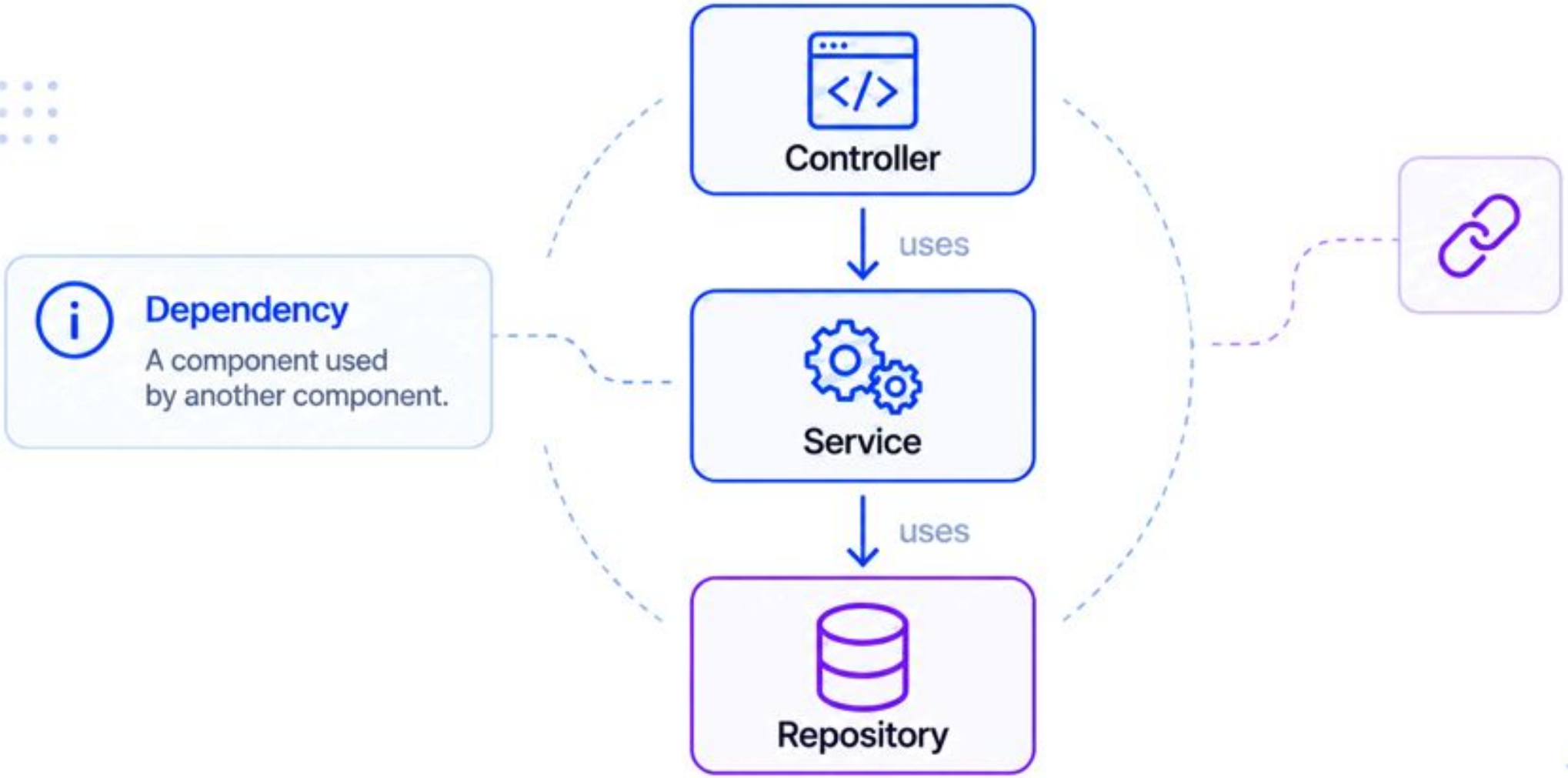
Дублирующая активная заявка
У пользователя уже есть активная заявка на этот ресурс

Проверка прав доступа
Только администратор может одобрить заявку

Ошибки как часть архитектуры



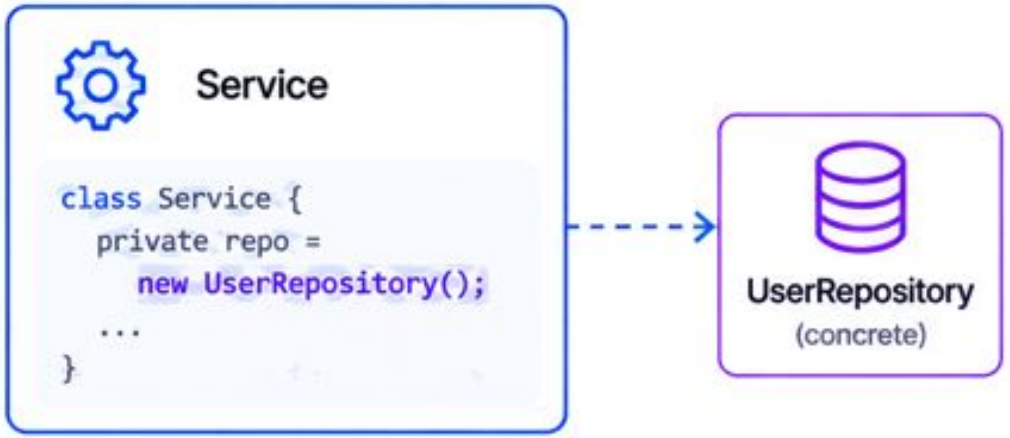
Dependencies



Dependency Injection

Bad

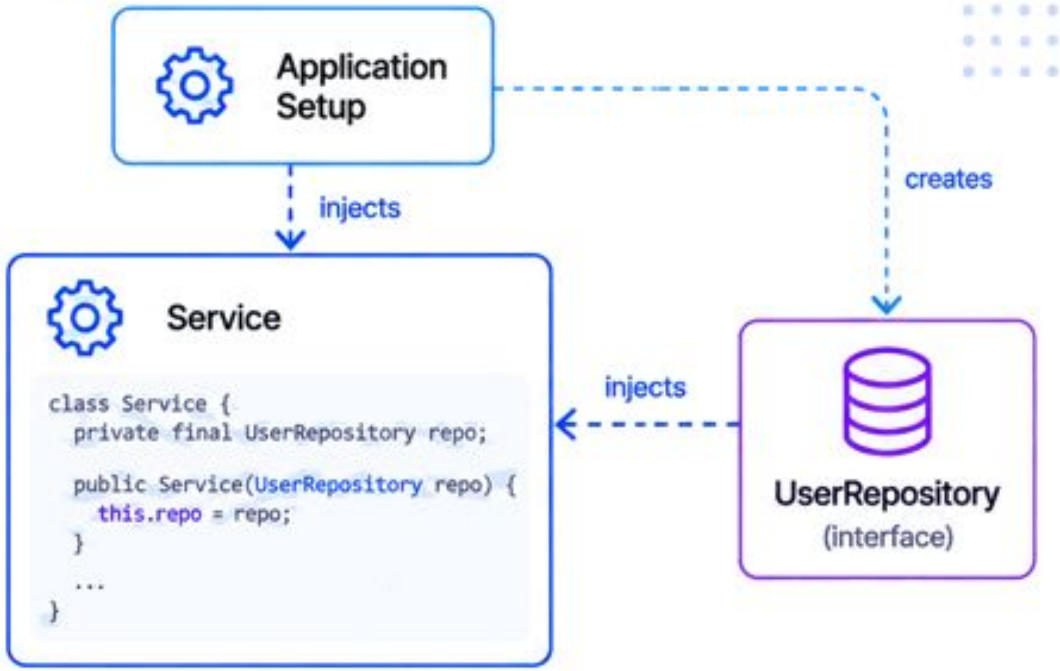
Service creates its own dependency



- Tightly coupled
- Hard to test
- Hard to change

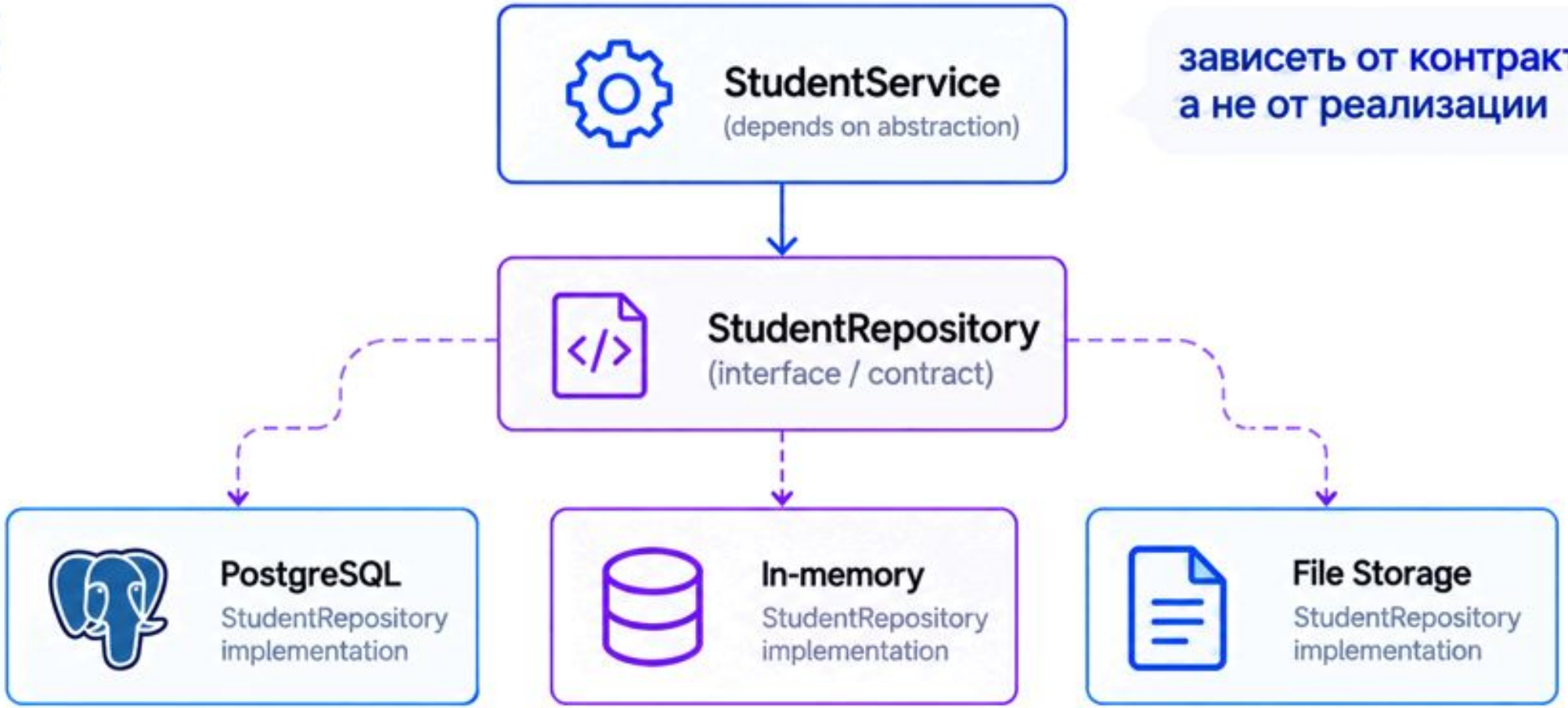
Good

Dependency is injected from outside



- Loosely coupled
- Easy to test
- Easy to change

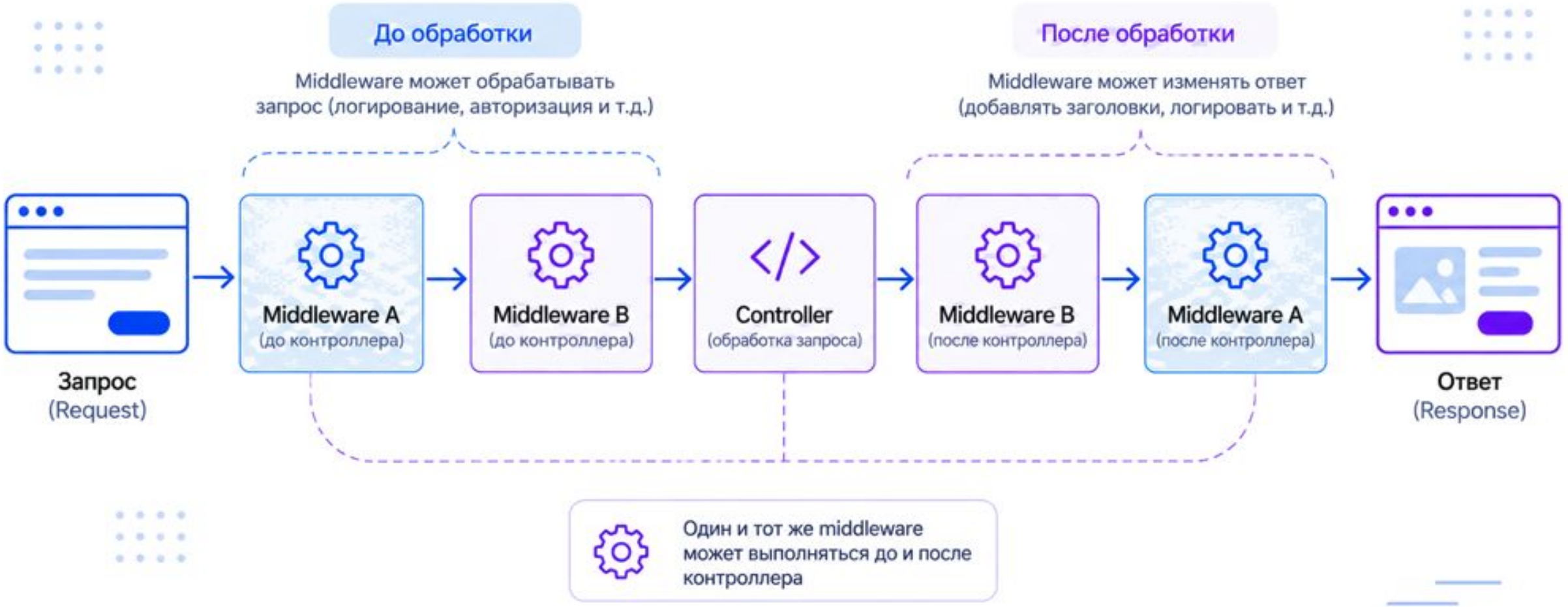
Dependency Inversion



Middleware: что это и зачем нужно



Middleware pipeline



Где middleware особенно полезен

ИТМО



Logging

Записывает информацию о запросах и событиях.



Authentication

Проверяет, кто делает запрос.



CORS

Управляет заголовками для кросс-доменных запросов.



Rate Limiting

Ограничивает количество запросов от клиента.



Metrics / Timing

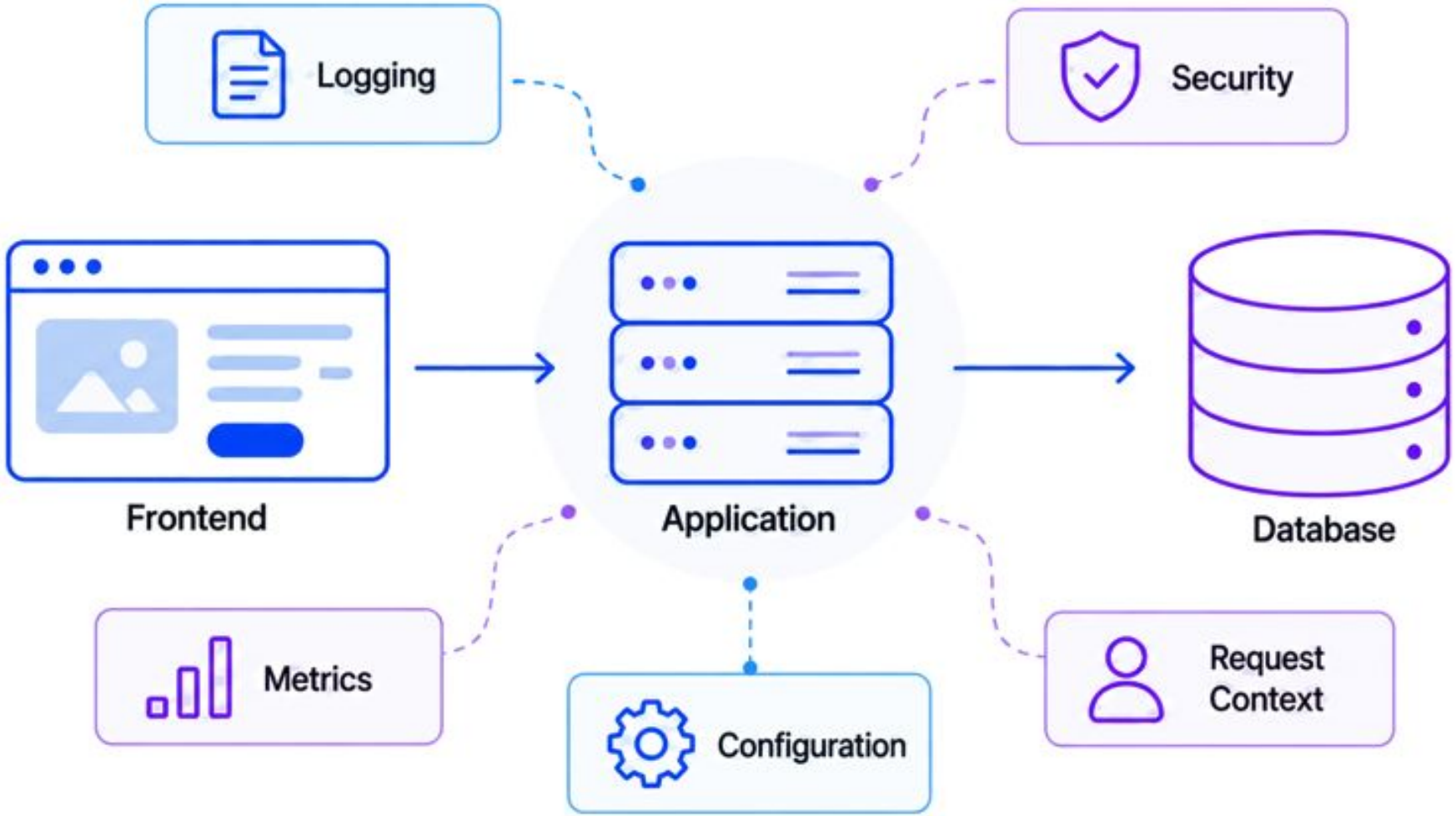
Измеряет время обработки запросов.



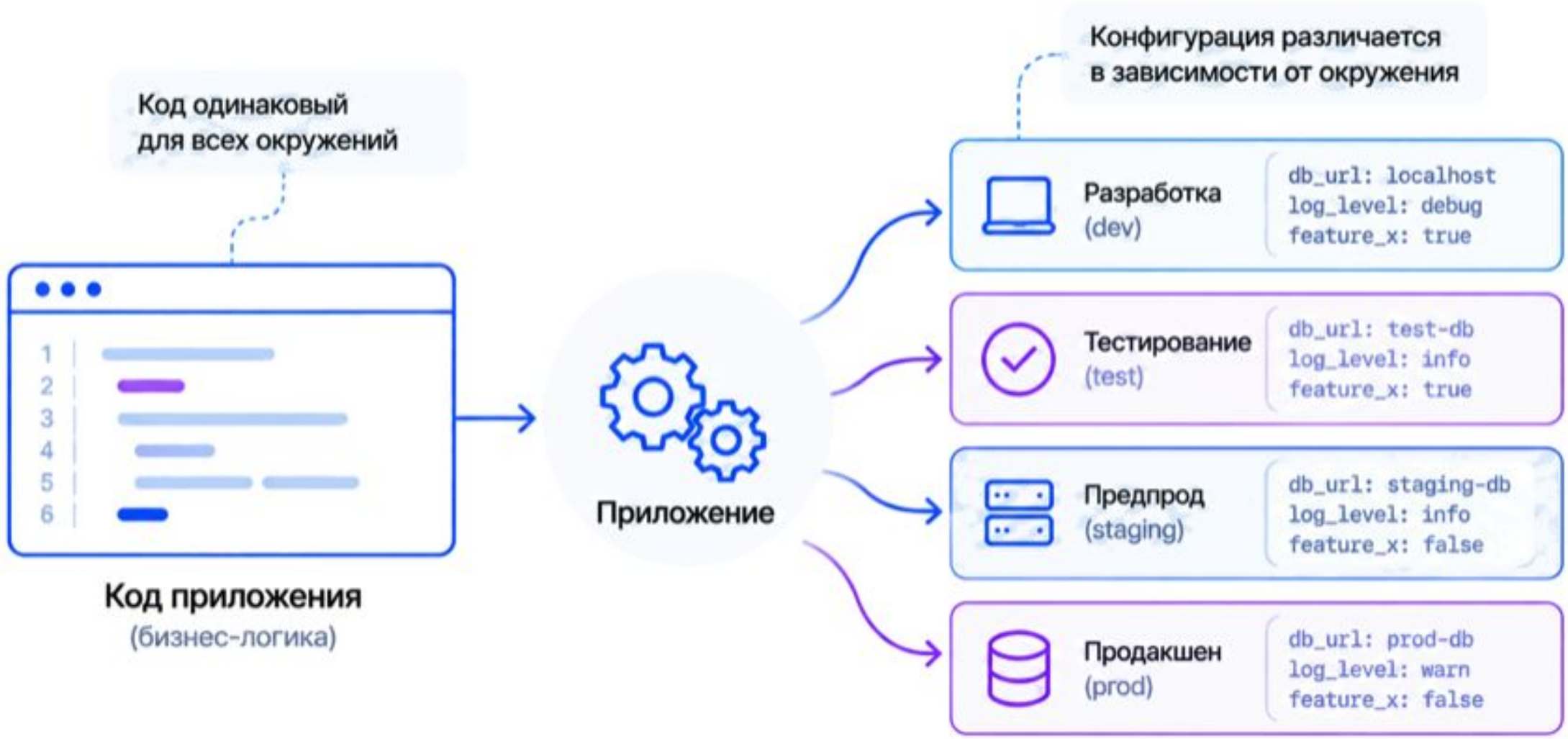
Error Handling

Обрабатывает ошибки и возвращает понятные ответы.

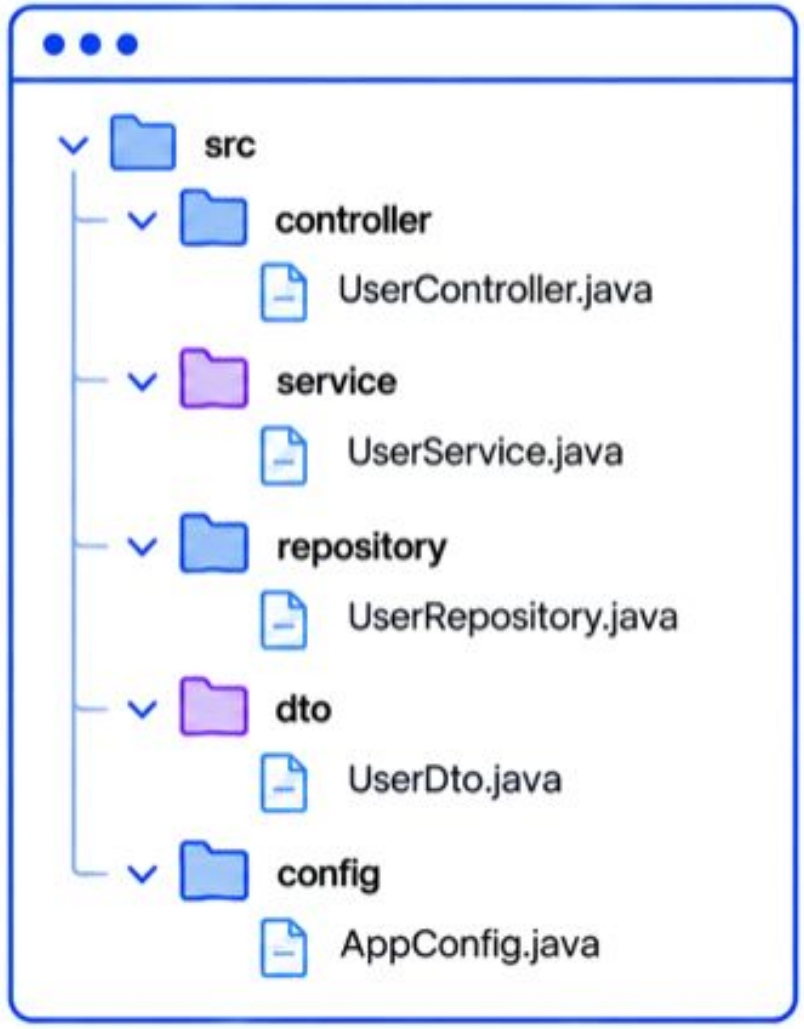
Cross-cutting Concerns



Configuration



Организация проекта: layer-based



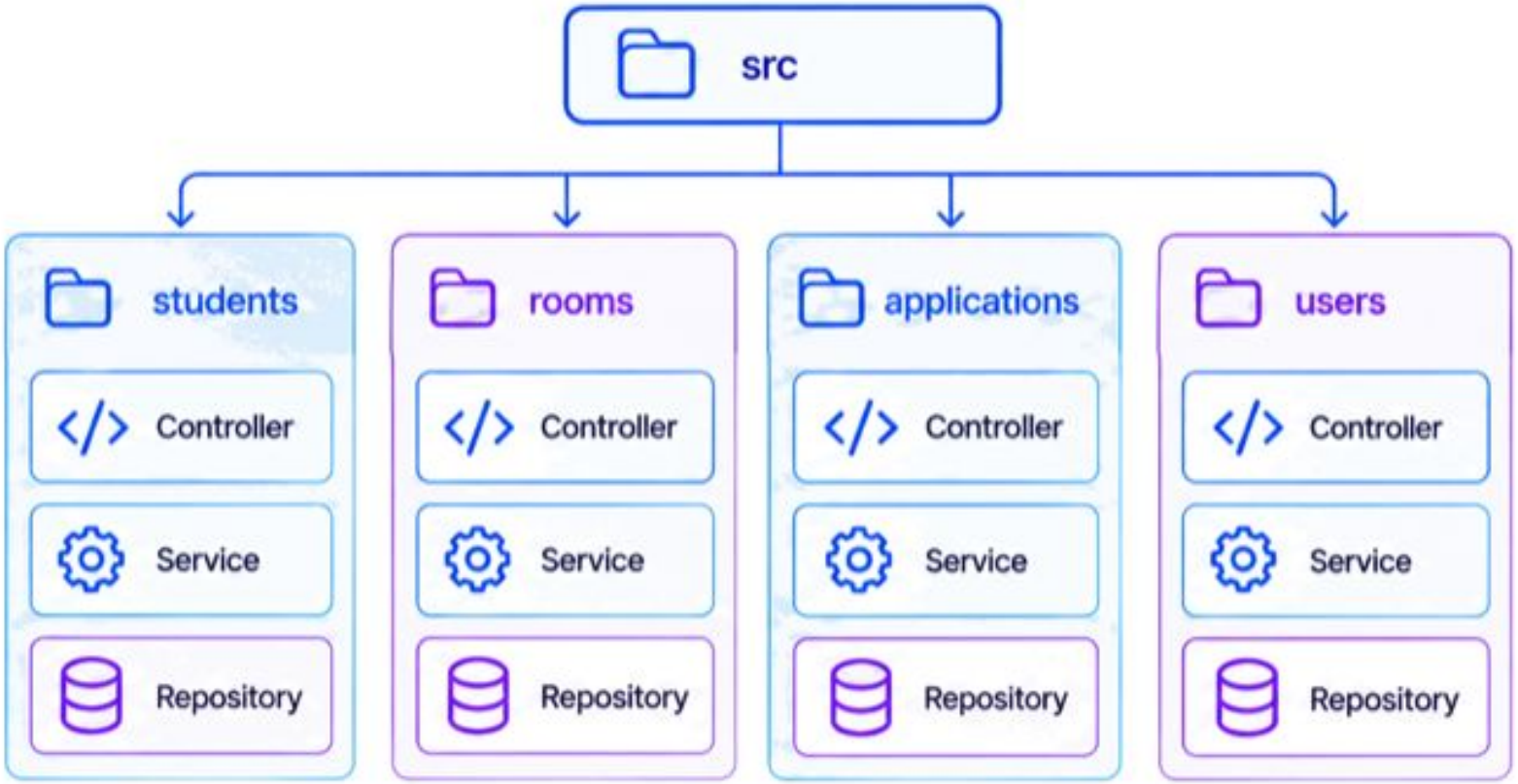
Плюсы

- Четкое техническое разделение слоев

Минусы

- Одна фича разбросана по разным папкам

Организация проекта: feature-based



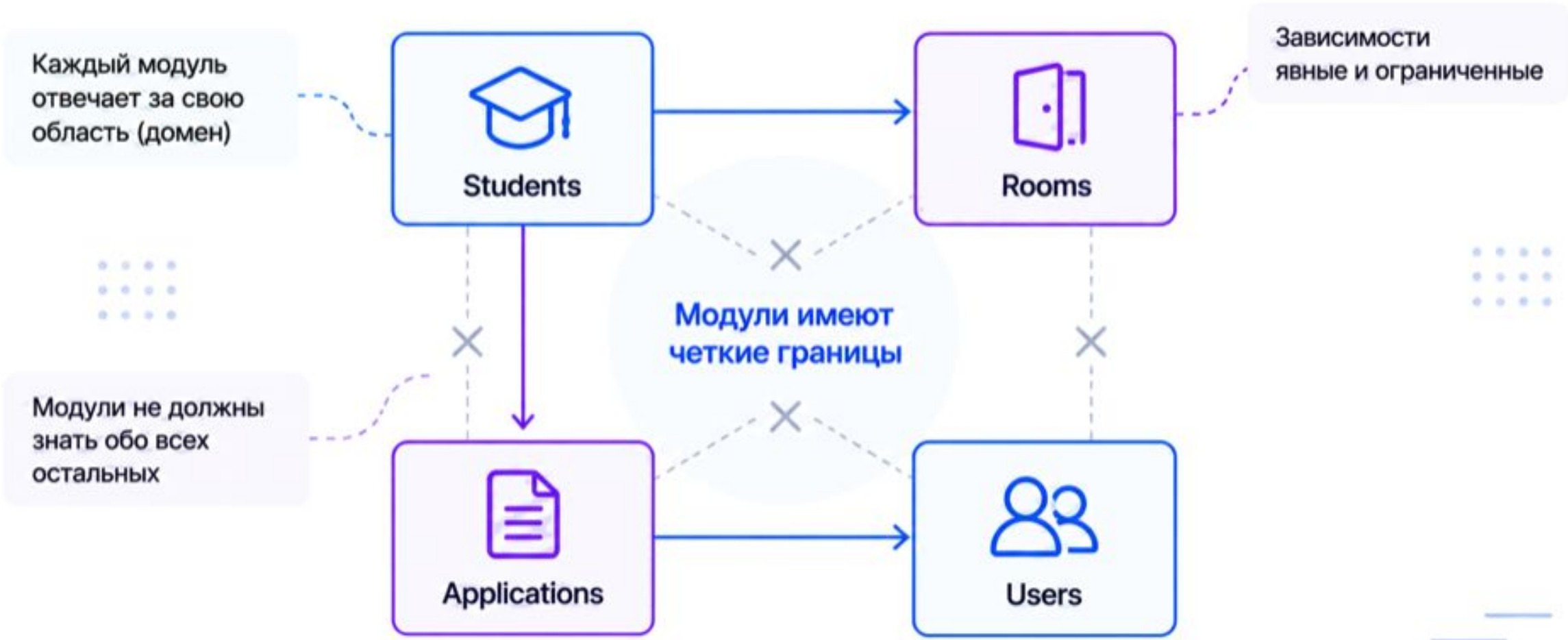
Плюсы

- Код фичи находится рядом
- Проще навигация внутри предметной области

Минусы

- Больше локальной структуры

Module Boundaries



Абстракции и границы



Антипаттерны архитектуры



Fat Controller

Вся бизнес-логика в контроллере



God Service

Один сервис делает слишком много



Circular Dependencies

Модули зависят друг от друга по кругу



Leaky Abstraction

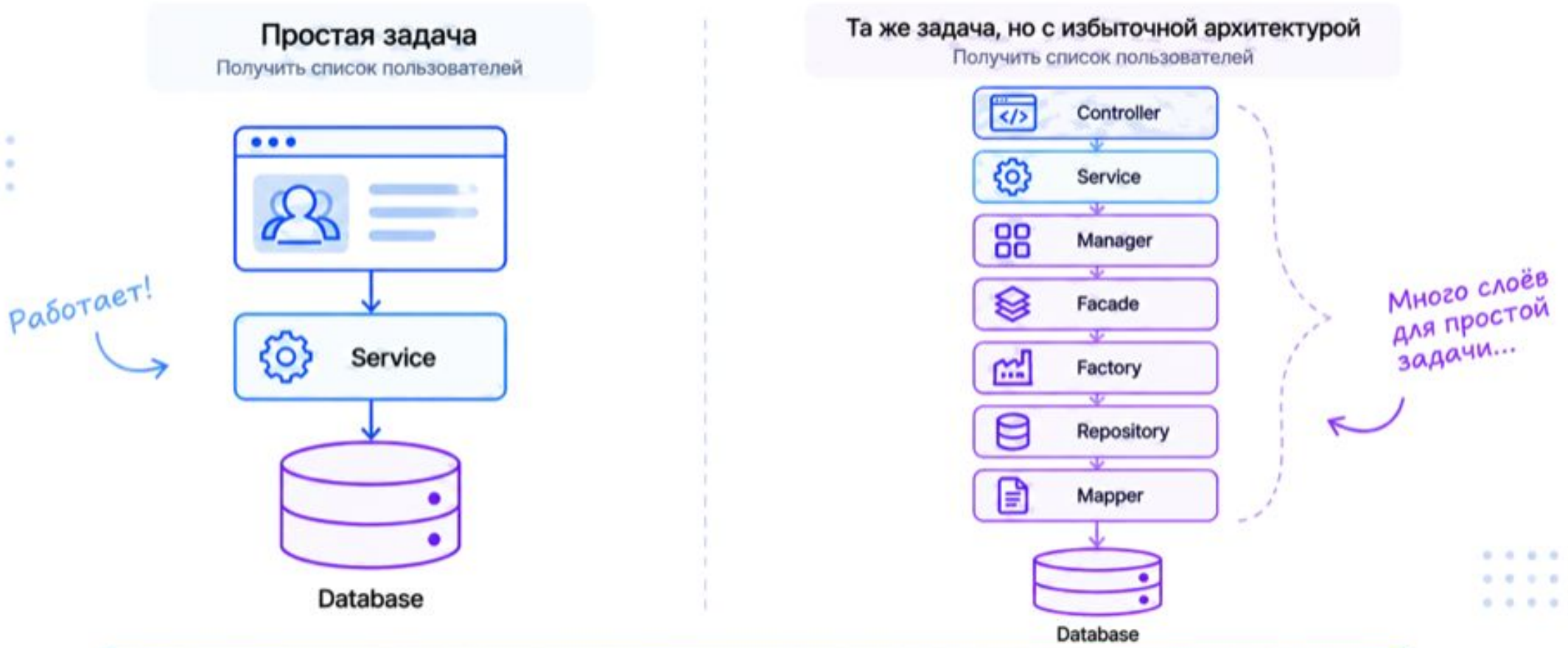
Детали реализации просачиваются наружу



Repository ради Repository

Использование репозитория без реальной необходимости

Overengineering



Дополнительная абстракция — это тоже стоимость.

Усложняйте архитектуру только тогда, когда это действительно даёт ценность.

Итоговая схема Backend



Главные выводы

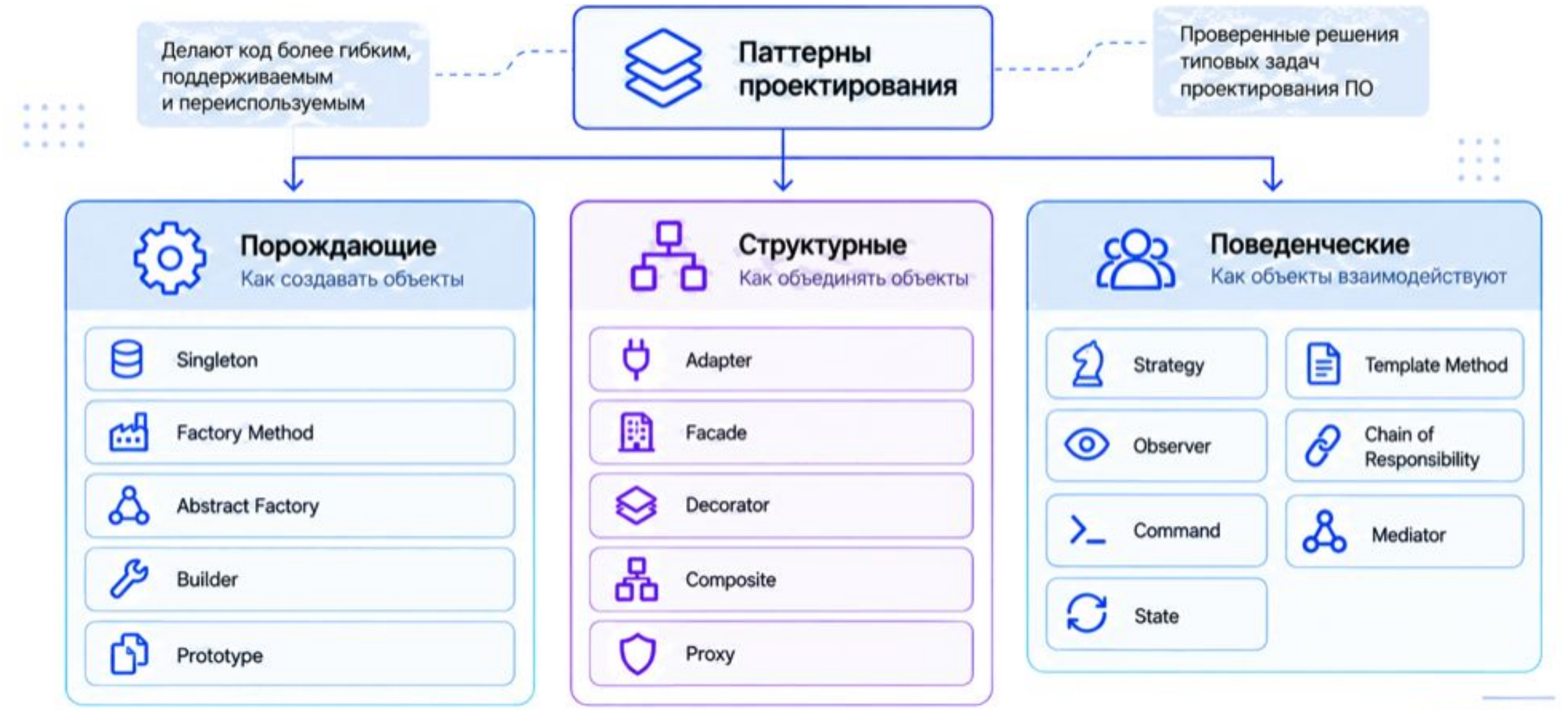
Границы
Каждый слой имеет чёткую роль и изолирован от деталей других слоёв

Ответственность
Один слой — одна зона ответственности

Зависимости
Зависим от абстракций (интерфейсов), а не конкретных реализаций

Паттерны проектирования: общая карта

ИТМО



Порождающие паттерны

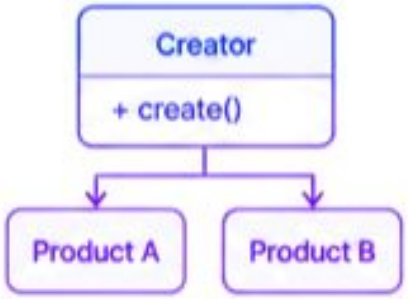
ИТМО

Singleton



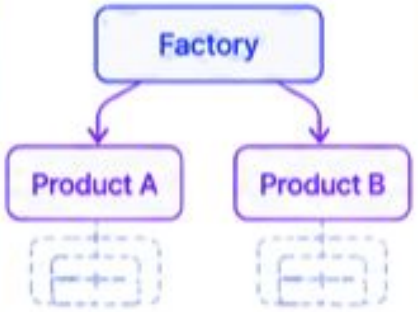
Когда нужен только один экземпляр класса.

Factory Method



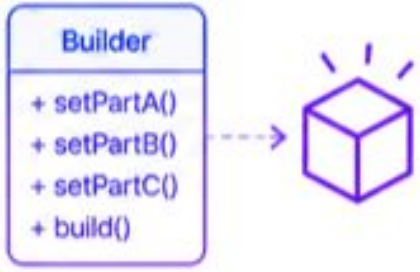
Когда подклассы должны сами решать, какой объект создавать.

Abstract Factory



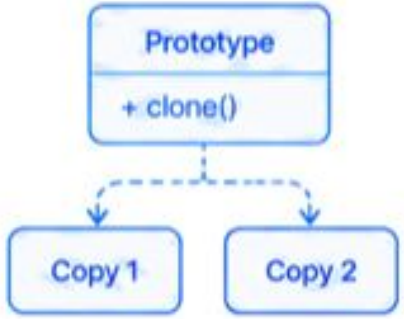
Когда нужно создавать семейства связанных объектов.

Builder



Когда объект сложный и его нужно собирать пошагово.

Prototype

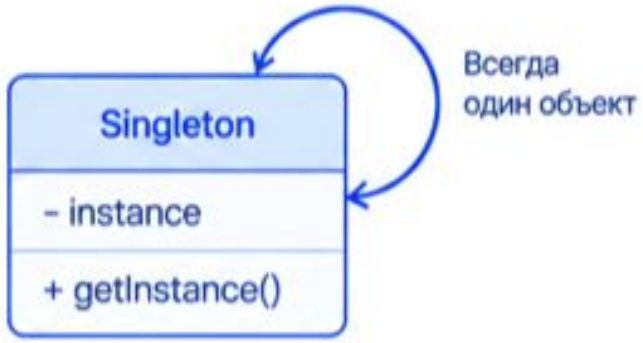


Когда нужны копии существующего объекта.

Singleton, Factory Method и Builder

Singleton

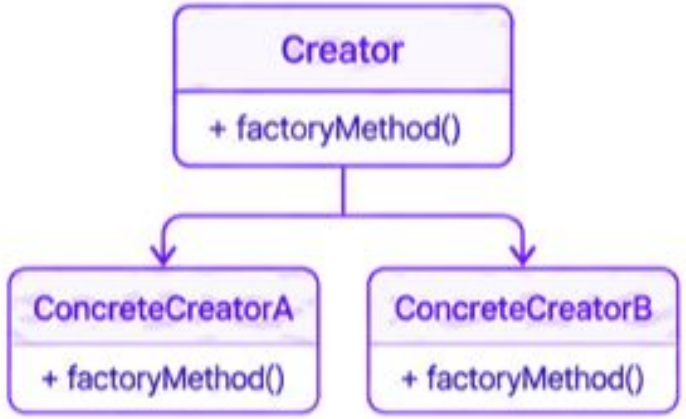
Один экземпляр класса



Когда нужен только один экземпляр класса и глобальная точка доступа к нему.

Factory Method

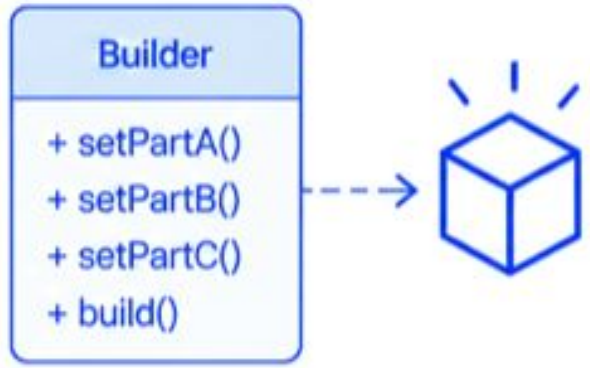
Создание через подкласс



Когда подклассы должны решать, какой конкретный объект создавать.

Builder

Пошаговое создание объекта



Когда объект сложно создавать и его нужно собирать поэтапно с разными вариантами конфигурации.

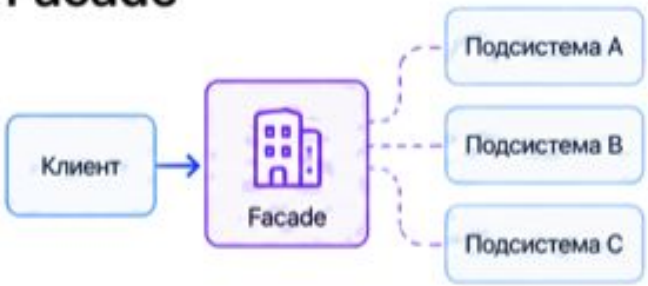
Структурные паттерны

Adapter



Преобразует интерфейс одного класса в интерфейс, ожидаемый клиентом.

Facade



Предоставляет простой интерфейс к сложной подсистеме.

Decorator



Динамически добавляет новые обязанности объекту.

Composite



Позволяет работать с одиночными объектами и их композициями единообразно.

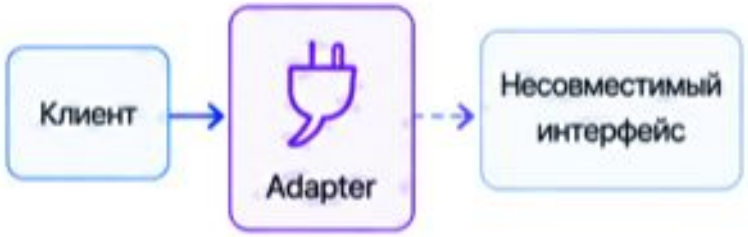
Proxy



Контролирует доступ к реальному объекту и может выполнять дополнительные действия.

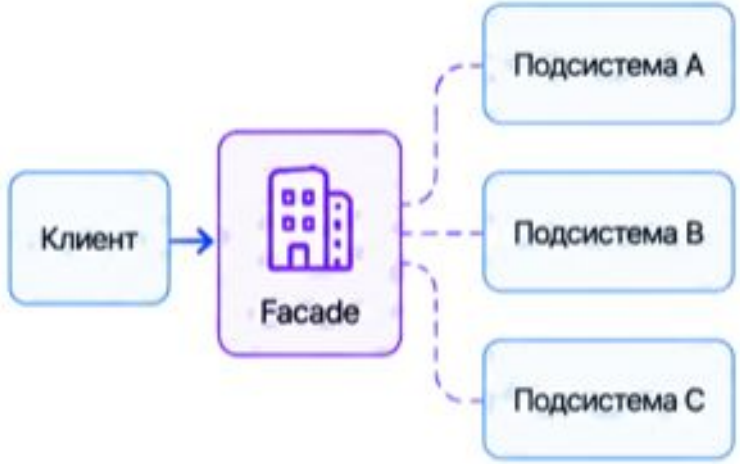
Adapter, Facade и Proxy

Adapter



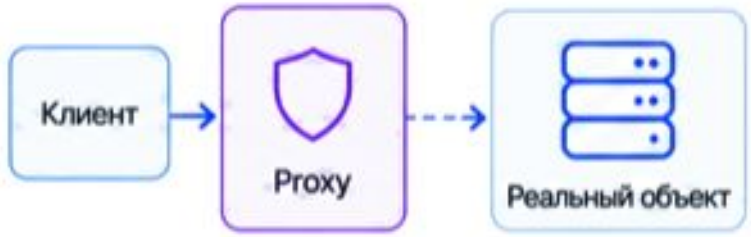
Преобразует интерфейс одного класса в другой интерфейс, ожидаемый клиентом.

Facade



Предоставляет простой интерфейс к сложной подсистеме.

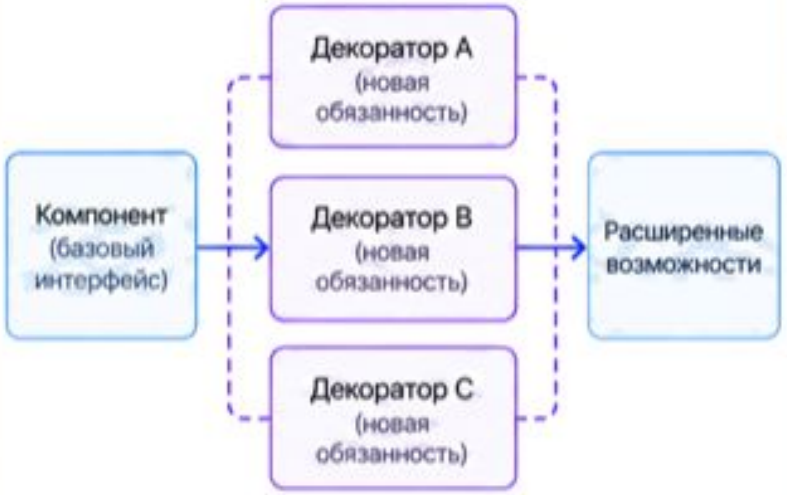
Proxy



Контролирует доступ к реальному объекту и может выполнять дополнительные действия.

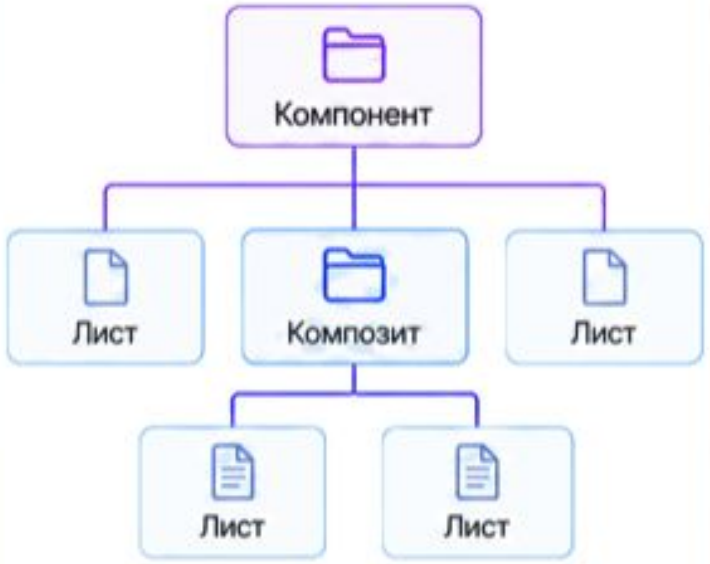
Decorator, Composite n Bridge

Decorator



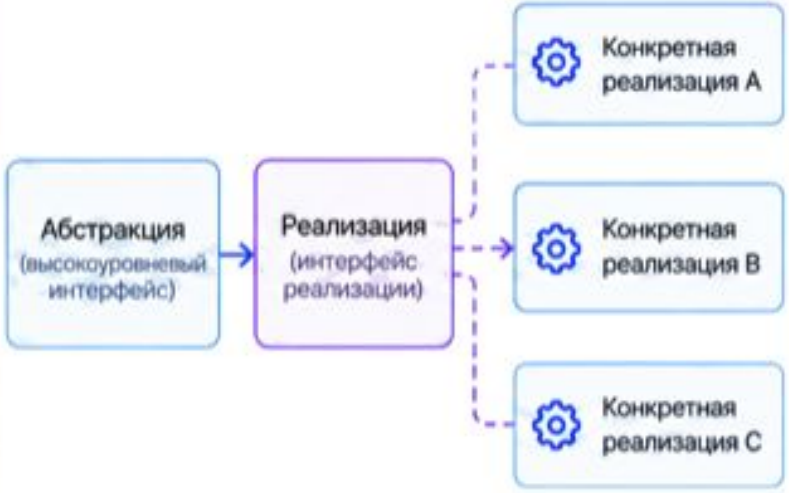
Динамически добавляет новые обязанности объекту, оборачивая его в декораторы. Позволяет расширять функциональность без изменения класса объекта.

Composite



Позволяет работать с одиночными объектами и их композициями единообразно. Объекты организуются в древовидную структуру, где композиты могут содержать другие компоненты.

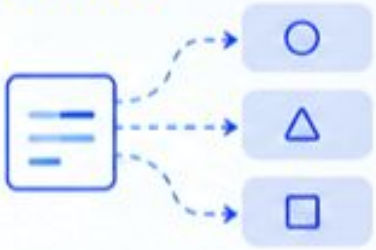
Bridge



Разделяет абстракцию и реализацию, позволяя им изменяться независимо друг от друга. Обеспечивает гибкость и расширяемость системы без фиксации на конкретных реализациях.

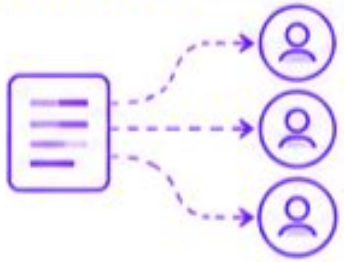
Поведенческие паттерны

Strategy



Позволяет выбирать алгоритм во время выполнения.

Observer



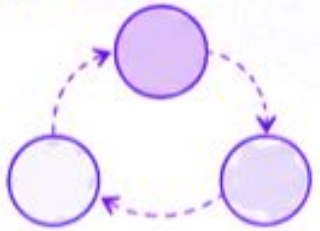
Позволяет объектам подписываться на события и получать уведомления.

Command



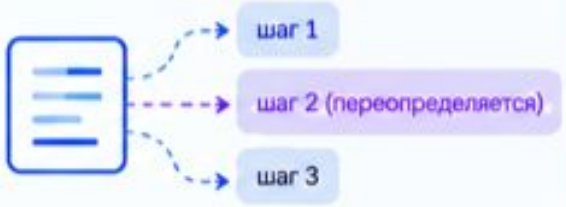
Инкапсулирует запрос как объект, позволяя откладывать и логировать его.

State



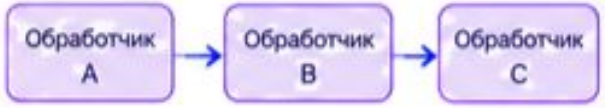
Позволяет объекту изменять поведение в зависимости от внутреннего состояния.

Template Method



Определяет скелет алгоритма, оставляя некоторые шаги для переопределения в подклассах.

Chain of Responsibility



Передает запрос по цепочке обработчиков, пока кто-то не сможет его обработать.

Mediator

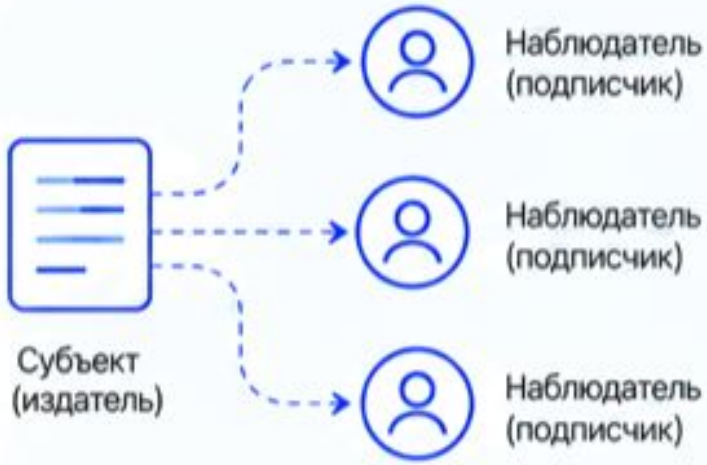


Централизует взаимодействие между объектами, уменьшая их связанность.

Observer, Command и Mediator

Observer

Подписка и уведомления



Позволяет объектам подписываться на события другого объекта и получать уведомления при изменениях его состояния.

Command

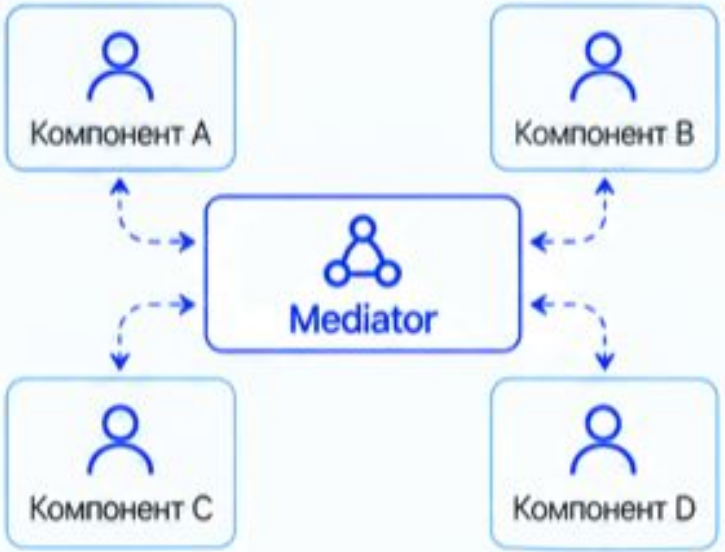
Запрос как объект



Инкапсулирует запрос в объект, позволяя передавать, ставить в очередь, логировать и откладывать его выполнение.

Mediator

Централизованное взаимодействие

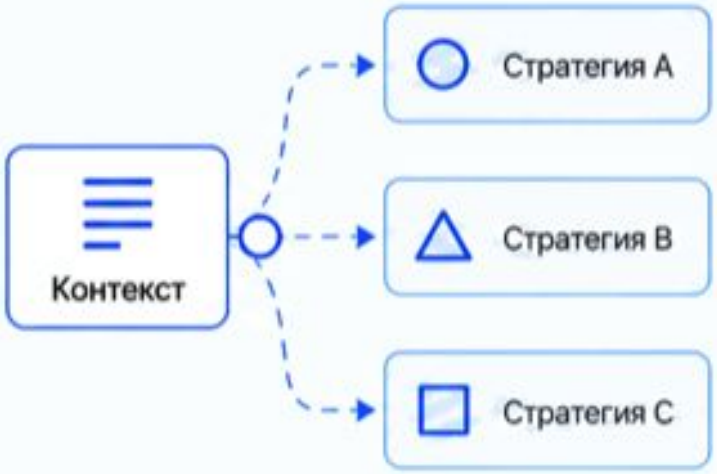


Централизует взаимодействие между объектами, уменьшая их связанность и упрощая управление взаимодействиями.

Strategy, State и Template Method

Strategy

Взаимозаменяемые алгоритмы



Позволяет выбирать алгоритм из набора взаимозаменяемых реализаций.

State

Поведение зависит от состояния



Позволяет объекту изменять поведение в зависимости от внутреннего состояния.

Template Method

Каркас алгоритма с настраиваемыми шагами



Определяет скелет алгоритма, оставляя некоторые шаги для переопределения в подклассах.

Chain of Responsibility в backend

ИТМО

Каждый обработчик может обработать запрос или передать его дальше

Цепочка обработчиков (Chain of Responsibility)

Гибкое добавление и изменение логики без изменения клиентов

Выполняет свою логику. Если не обработал — передаёт дальше.

Каждый обработчик независим и не знает о других.

При ошибке может прервать цепочку и вернуть ответ.

Перехватывает ошибки на любом этапе цепочки и формирует ответ.

Запрос (Request)

Middleware A
Может обработать или передать дальше

Middleware B
Может обработать или передать дальше

Validation
Может обработать или передать дальше

Controller
Обрабатывает запрос и формирует ответ

Error Handler
Обрабатывает ошибки (если они возникли)

Ответ (Response)

Типичные сценарии использования

Логирование
Запись информации о запросах

Аутентификация
Проверка пользователя и токена

Валидация
Проверка входных данных

Ограничение частоты запросов
Throttling / Rate limiting

Обработка ошибок
Единая обработка и формат ошибок

Паттерны в backend-архитектуре

